

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ

Факультет комп'ютерних наук
Кафедра програмної інженерії

КВАЛІФІКАЦІЙНА РОБОТА
освітнього ступеня «бакалавр»

**на тему: «Програмна система для безконтактного замовлення їжі в ресторані
з використанням QR-технологій»**

Виконав: студент групи ПЗПІ-22-1
Денис ТОКАР

Керівник: проф. Зоя ДУДАР

Харків 2026

РЕФЕРАТ

Кваліфікаційна робота: 83 с., 13 рис., 2 табл., 10 джерел.

Об'єкт дослідження — процес автоматизації обслуговування клієнтів у закладах громадського харчування малого та середнього формату.

Предмет дослідження — методи та засоби розробки програмної системи для безконтактного замовлення їжі на основі QR-технологій із використанням технологічного стеку ReactJS, Node.js та MongoDB.

Мета роботи — проєктування та розробка програмної системи для безконтактного замовлення їжі в ресторані з використанням QR-технологій, що забезпечує повний цикл взаємодії гостя із закладом від сканування QR-коду до завершення оплати без обов'язкового залучення обслуговуючого персоналу.

Методи дослідження — порівняльний аналіз існуючих програмних рішень у сфері автоматизації ресторанного обслуговування; методології формування вимог до програмного забезпечення відповідно до стандарту IEEE Std 830-1998; методи об'єктно-орієнтованого проєктування; шаблони архітектурного проєктування веб-застосунків (трирівнева архітектура, multi-tenant SaaS, шарова організація серверного коду); методи проєктування документно-орієнтованих баз даних; принципи проєктування REST API та протоколу WebSocket для передачі даних у режимі реального часу.

У результаті виконання роботи спроектовано та реалізовано клієнт-серверну систему, що складається з односторінкового веб-застосунку для гостей і персоналу та серверної частини з підтримкою REST API та WebSocket-з'єднань. Сформовано повний комплект функціональних та нефункціональних вимог, що містить 15 функціональних модулів, понад 80 функціональних вимог із унікальними ідентифікаторами та понад 70 критеріїв приймання у форматі Given/When/Then. Спроектовано схему документно-орієнтованої бази даних з 23 колекціями. Реалізовано інтеграцію з зовнішніми сервісами: платіжний шлюз LiqPay, Google

OAuth 2.0, AWS S3, Resend, Google Translate API. Реалізовано модель монетизації Freemium із технічним розподілом функціоналу між безкоштовним та преміум-тарифами на рівні бази даних та проміжного програмного забезпечення.

Практична цінність роботи полягає у створенні програмного продукту, придатного для реального впровадження в закладах громадського харчування малого та середнього формату. Модульна побудова системи забезпечує її гнучкість та можливість подальшого розширення функціональності відповідно до потреб конкретного підприємства. Запропонована модель монетизації Freemium робить продукт фінансово доступним для закладів малого формату, що було визначено як ключова ринкова прогалина за результатами аналізу предметної галузі.

Ключові слова: QR-ТЕХНОЛОГІЇ, БЕЗКООНТАКТНЕ ЗАМОВЛЕННЯ, АВТОМАТИЗАЦІЯ РЕСТОРАНУ, SAAS, ОДНОСТОПІНКОВИЙ ВЕБЗАСТОСУНОК, REACTJS, NODE.JS, MONGODB, WEBSOCKET, REST API, BYOD, FREEMIUM, KDS.

ЗМІСТ

ВСТУП	9
1 АНАЛІЗ ПРЕДМЕТНОЇ ГАЛУЗІ.....	11
1.1 Загальна характеристика предметної галузі.....	11
1.2 Огляд існуючих програмних рішень.....	12
1.2.1 Poster POS	12
1.2.2 OddMenu	12
1.2.3 Choice QR.....	13
1.2.4 Kavapp QR.....	14
1.3 Порівняльний аналіз існуючих рішень	14
1.4 Концепція системи та об'єкт автоматизації.....	16
1.5 Рольова модель системи.....	16
1.6 Функціональні модулі системи	17
1.7 Позиціонування на ринку.....	19
1.8 Модель монетизації	20
1.9 Обґрунтування вибору технологічного стеку	21
2 ФОРМУВАННЯ ВИМОГ	23
2.1 Огляд функціональних вимог	23
2.1.1 Модуль QR-менеджменту та ідентифікація столика	23
2.1.2 Модуль меню: інгредієнти, модифікатори та добавки	25
2.1.3 Модуль формування замовлення: кошик та групи подачі	27
2.1.4 Модуль управління замовленням столику	28
2.1.5 Модуль передачі замовлення на кухню (KDS)	30

	5
2.1.6 Модуль виклику офіціанта.....	31
2.1.7 Модуль відстеження статусу замовлення	32
2.1.8 Модуль оплати	33
2.1.9 Модуль аналітики та звітності.....	34
2.1.10 Модуль генератора PDF-меню	35
2.1.11 Модуль системи відгуків.....	36
2.1.12 Модуль авторизації та облікових записів.....	37
2.1.13 Модуль управління меню.....	38
2.1.14 Модуль управління персоналом та onboarding ресторану.....	39
2.1.15 Модуль офлайн-режиму клієнтського інтерфейсу (PWA)	41
2.2 Огляд нефункціональних вимог	43
2.2.1 Вимоги до продуктивності.....	43
2.2.2 Вимоги до безпеки	43
2.2.3 Атрибути якості.....	43
2.2.4 Вимоги до сумісності	44
2.2.5 Вимоги до логування	44
2.2.6 Вимоги до моніторингу	44
2.3 Use Case діаграми системи за ролями.....	44
2.3.1 Use Case діаграма ролі «Гість».....	44
2.3.2 Use Case діаграма ролі «Офіціант»	45
2.3.3 Use Case діаграма ролі «Кухар»	45
2.3.4 Use Case діаграма ролі «Адміністратор».....	46
2.4 Вимоги до тарифної моделі	46

3	АРХІТЕКТУРА ТА ПРОЄКТУВАННЯ	48
3.1	Загальна архітектура системи.....	48
3.2	Архітектура клієнтської частини	49
3.2.1	Технологічний стек та інструменти збірки	49
3.2.2	Структурна організація компонентів	50
3.2.3	Управління станом через контексти React	50
3.2.4	Архітектура WebSocket-клієнта	51
3.2.5	Підтримка PWA та офлайн-режиму.....	51
3.3	Архітектура серверної частини	52
3.3.1	Технологічний стек та модульна організація.....	52
3.3.2	REST API та принципи проєктування	53
3.3.3	Middleware-конвеєр обробки запитів.....	53
3.3.4	Реалізація розподілу тарифів через middleware.....	54
3.4	Проєктування бази даних.....	54
3.4.1	Обґрунтування вибору MongoDB	54
3.4.2	Огляд ключових колекцій	55
3.4.3	Стратегія цілісності даних	56
3.4.4	Діаграма стану замовлення	56
3.5	Архітектура взаємодії в реальному часі	57
3.5.1	Модель кімнат та підписок	57
3.5.2	Каталог подій та формат повідомлень.....	57
3.5.3	Механізм event replay та відновлення з'єднання.....	58
3.6	Інтеграція з зовнішніми сервісами.....	58

3.6.1 Інтеграція з платіжним шлюзом LiqPay	58
3.6.2 Інтеграція з Google OAuth 2.0.....	59
3.6.3 Інтеграція з AWS S3 для зберігання зображень	59
3.6.4 Інтеграція з Resend для транзакційних email	60
3.6.5 Інтеграція з Google Translate API	60
3.7 Безпека та автентифікація	60
3.7.1 Модель автентифікації	60
3.7.2 Авторизація через RBAC	61
3.7.3 Захист від типових атак.....	61
3.7.4 Зберігання паролів та секретів.....	61
3.8 Проєктування інтерфейсу користувача	62
3.9 Сценарії взаємодії компонентів системи.....	63
3.9.1 Повний цикл обслуговування гостя.....	63
3.9.2 Обробка помилок платіжного шлюзу	63
3.9.3 Відновлення WebSocket-з'єднання та event replay.....	64
4 ОПИС ПРИЙНЯТИХ ПРОГРАМНИХ РІШЕНЬ.....	65
4.1 Структура коду проєкту	65
4.2 Управління станом через контексти React	65
4.3 Архітектура єдиного WebSocket-з'єднання.....	66
4.4 Підтримка багатомовності	67
4.5 Реалізація тарифних обмежень.....	68
4.6 Офлайн-черга замовлень	69
4.7 Система сповіщень персоналу за ролями.....	70

	8
4.8 Генератор PDF-меню	70
5 ТЕСТУВАННЯ	72
5.1 Стратегія тестування	72
5.2 Інструменти тестування	72
5.3 Тестові дані	73
5.4 Тестування за критеріями приймання	73
5.5 Результати тестування	74
6 ВПРОВАДЖЕННЯ	76
6.1 Середовище розгортання	76
6.2 Конфігурація через змінні оточення	76
6.3 Збірка та розгортання	77
6.4 Моніторинг та логування	77
ВИСНОВКИ	79
ПЕРЕЛІК ДЖЕРЕЛ ПОСИЛАНЬ	82

ВСТУП

Розвиток інформаційних технологій та стрімке їх поширення суттєво змінюють підходи до організації обслуговування в закладах харчування. Сучасний споживач очікує швидкого та зручного обслуговування з мінімальним залученням персоналу, що формує попит на цифрові рішення, здатні замінити або доповнити традиційну модель роботи ресторану [1].

Існуюча практика прийому замовлень через офіціанта супроводжується низкою проблем: затримками в години пік, людськими помилками, витратами на друк і оновлення паперових меню, відсутністю інструментів для збору та аналізу даних про поведінку клієнтів. Зазначені чинники негативно впливають як на якість клієнтського досвіду, так і на економічну ефективність закладу [1, 2].

QR-технології є доступним і технологічно зрілим інструментом, що дозволяє усунути більшість із перелічених недоліків без значних інфраструктурних витрат з боку закладу [2]. Розміщення QR-коду на столику надає гостю можливість самостійно переглянути меню, сформувати та відправити замовлення безпосередньо на кухню, викликати офіціанта та здійснити оплату — все це в межах єдиного веб-інтерфейсу без встановлення додаткового програмного забезпечення.

Метою даної кваліфікаційної роботи є проєктування та розробка програмної системи для безконтактного замовлення їжі в ресторані з використанням QR-технологій.

Для досягнення поставленої мети необхідно вирішити такі завдання:

- дослідити предметну галузь та проаналізувати наявні програмні рішення у сфері автоматизації ресторанного обслуговування;
- сформувати функціональні та нефункціональні вимоги до системи;
- спроектувати архітектуру програмної системи та структуру бази даних;
- реалізувати клієнтський інтерфейс у вигляді SPA для гостей ресторану та панелі керування для персоналу;

- розробити серверну частину на основі REST API з підтримкою WebSocket для передачі замовлень у режимі реального часу;

- реалізувати розширені модулі системи: аналітику продажів, генерацію PDF-меню, систему зворотного зв'язку, CRM-елементи та фінансовий модуль з підтримкою онлайн-оплати і розділення рахунку.

Об'єктом дослідження є процес автоматизації обслуговування клієнтів у закладах громадського харчування.

Предметом дослідження є методи та засоби розробки програмної системи на основі QR-технологій з використанням стеку ReactJS, Node.js та MongoDB.

Практична цінність роботи полягає у створенні програмного продукту, придатного для реального впровадження в закладах громадського харчування. Модульна побудова системи забезпечує її гнучкість та можливість подальшого розширення функціональності відповідно до потреб конкретного підприємства.

1 АНАЛІЗ ПРЕДМЕТНОЇ ГАЛУЗІ

1.1 Загальна характеристика предметної галузі

Ресторанний бізнес належить до галузей із високою операційною інтенсивністю, де швидкість і точність обробки замовлень безпосередньо впливають на задоволеність клієнтів та фінансовий результат підприємства. Традиційна схема обслуговування передбачає послідовний ланцюжок взаємодій: гість — офіціант — кухня — офіціант — гість, кожна ланка якої є потенційним джерелом затримок і помилок [3].

Цифровізація ресторанного сервісу розвивається за кількома напрямками: автоматизація внутрішніх процесів (системи точок продажу — POS-системи, системи управління кухнею — KDS), діджиталізація взаємодії з клієнтом (мобільні застосунки, самообслуговування) та безконтактне обслуговування на основі QR-технологій. Останній напрям набув особливого поширення після 2020 року і на сьогодні є одним із найбільш доступних інструментів цифрової трансформації для малого та середнього бізнесу у сфері готельно-ресторанного бізнесу (HoReCa) [4].

QR-код (Quick Response Code) — двовимірний штрих-код, що кодує текстову інформацію, зокрема URL-адресу. Сканування QR-коду вбудованою камерою смартфона без встановлення додаткових застосунків є ключовою перевагою цієї технології з точки зору доступності для кінцевого користувача. Розміщення індивідуального QR-коду на кожному столику ресторану дозволяє системі автоматично ідентифікувати місце розташування гостя та прив'язати замовлення до конкретного столика [3].

Технологічну основу сучасних систем безконтактного замовлення складають:

- прогресивні вебзастосунки (PWA) або адаптивні односторінкові застосунки (SPA), що не потребують встановлення;
- програмні інтерфейси застосунків (REST API або GraphQL) для обміну даними між клієнтом і сервером;

- протоколи двостороннього зв'язку (WebSocket) для передачі подій у режимі реального часу;
- хмарні або локальні реляційні та документно-орієнтовані бази даних (БД) для зберігання меню, замовлень і аналітичних даних.

1.2 Огляд існуючих програмних рішень

1.2.1 Poster POS

Poster POS — хмарна система автоматизації ресторану українського походження, що охоплює функції POS-термінала, складського обліку, аналітики та управління персоналом [5]. Система надає можливість створення QR-меню для гостей, однак ця функція реалізована як статичний перегляд позицій без повноцінного циклу замовлення безпосередньо з боку клієнта. Прийом замовлень залишається прерогативою персоналу через планшет або касовий термінал.

З точки зору аналітики Poster POS пропонує розвинений дашборд із показниками виручки, популярності страв та ефективності персоналу. Проте система є комплексним корпоративним продуктом із щомісячною абонентською платою, що робить її фінансово недоступною для малих закладів. Відкритого API для глибокого налаштування клієнтської частини система не надає, що унеможливорює адаптацію інтерфейсу під фірмовий стиль конкретного закладу [5].

1.2.2 OddMenu

OddMenu — спеціалізований SaaS-сервіс для створення цифрових QR-меню ресторанів [6]. На відміну від Poster POS, продукт зосереджений виключно на взаємодії з гостем: сканування QR-коду відкриває адаптивне меню з фотографіями, описами та фільтрацією за категоріями. Інтерфейс оптимізований для мобільних пристроїв і не потребує реєстрації з боку клієнта.

Однак OddMenu є, по суті, інструментом для відображення меню, а не повноцінною системою замовлення [6]. Платформа не підтримує відправлення

замовлення на кухню в режимі реального часу, не має модуля онлайн-оплати, аналітики, системи зворотного зв'язку чи виклику офіціанта. Функціонал розділення рахунку та CRM-елементи також відсутні. Сервіс надає обмежені можливості для брендування інтерфейсу в рамках безкоштовного тарифу.

1.2.3 Choice QR

Choice QR — хмарна платформа, що надає ресторанам інструменти для управління взаємодією з гостями через QR-оплату, замовлення до столика, цифрове меню, бронювання та CRM для гостей [7]. Відмінністю від більшості конкурентів є те, що гості можуть самостійно розмістити або доповнити замовлення у будь-який момент без очікування, а заклад отримує миттєві сповіщення про нові замовлення, які можуть бути одразу передані на кухню.

Платформа також реалізує функцію оплати через QR-код із підтримкою розділення рахунку між гостями: кожен може самостійно обрати позиції для оплати та залишити чайові. При цьому гостям не потрібно реєструватися або встановлювати застосунок. Також є можливість додати функціонал резервації столиків. З точки зору CRM, система збирає клієнтську базу з усіх каналів, підтримує відправлення промоакцій, фільтрацію клієнтів і перегляд зворотного зв'язку, що, за даними самої платформи, забезпечує приріст виручки до 35% [7].

На додаток ця система надає можливість інтеграції з популярними сервісами доставки. А також платформа дає змогу інтегрувати схожі популярні POS системи, зокрема Syrve, Poster, Servio та інші, зі збереженням накопичених даних та автоматичною синхронізацією замовлень.

Проте основним недоліком Choice QR є обмежений функціонал в безкоштовному та найнижчих тарифах, зокрема відкритий API, CRM, пріоритетна підтримка, інтеграція з POS системами та власний домен, доступні лише на вищих тарифних планах [7]. Генерація PDF-меню для фізичного друку в системі відсутня.

1.2.4 Kavapp QR

Kavapp — українська система автоматизації, що охоплює продажі, склад, логістику, персонал і фінанси для закладів харчування та магазинів, синхронізована через три спеціалізовані застосунки: для менеджменту й аналізу, для продажів і фіскалізації, а також для управління логістикою та складом [8].

Серед маркетингових інструментів платформа включає QR-меню, програми лояльності та знижки. Проте QR-меню в Kavapp реалізоване як допоміжний інструмент у межах ширшої POS-системи, а не як самостійний канал взаємодії гостя з кухнею [8]. Самостійне оформлення замовлення клієнтом через QR-інтерфейс без участі персоналу не передбачено. Система орієнтована насамперед на автоматизацію роботи персоналу, а не на безконтактний клієнтський досвід. Функції виклику офіціанта через інтерфейс, розділення рахунку та генерації PDF-меню відсутні.

1.3 Порівняльний аналіз існуючих рішень

У таблиці 1.1 наведено порівняння розглянутих систем за ключовими функціональними та технічними характеристиками.

Таблиця 1.1 — Порівняльний аналіз існуючих рішень

Характеристика	Poster POS	OddMenu	Choice QR	Kavapp QR	Система, що розробляється
QR-меню для гостей	Так	Так	Так	Так	Так
Самостійне замовлення гостем	Через персонал	Ні	Так	Ні	Так
Передача замовлення на кухню в реальному часі	Через персонал	Ні	Так	Через персонал	Так
Виклик офіціанта через інтерфейс	Ні	Ні	Так	Ні	Так

Редагування замовлення «на ходу»	Ні	Ні	Так	Ні	Так
Онлайн-оплата	Так	Ні	Так	Ні	Так
Розділення рахунку (split bill)	Ні	Ні	Так	Ні	Ні
Аналітика та звітність	Розширена	Відсутня	Розширена	Базова	Базова
Генерація PDF-меню	Ні	Ні	Ні	Ні	Так
Система зворотного зв'язку	Частково	Ні	Так	Частково	Так
CRM / історія замовлень	Частково	Так	Частково	Так	Так
Орієнтація на малий бізнес	Частково	Так	Частково	Так	Так

Проведений аналіз дозволяє розподілити розглянуті рішення на три умовні категорії. До першої належать комплексні POS-платформи (Poster POS, Kavapp), які мають розвинений внутрішній функціонал для персоналу, але не забезпечують повноцінного самостійного замовлення гостем через QR-інтерфейс і є функціонально та фінансово надлишковими для закладів малого і середнього розміру. До другої категорії відносяться вузькоспеціалізовані сервіси цифрового меню (OddMenu), що є доступними у впровадженні, проте обмежені виключно функцією відображення інформації. Третю категорію формують повнофункціональні QR-платформи (Choice QR), які найближчі до концепції системи, що розробляється, однак орієнтовані на міжнародний ринок та потребують значних витрат на підключення.

Таким чином, проведений аналіз виявив прогалину на ринку: існуючі рішення або фінансово недоступні для малого бізнесу через надлишковий функціонал (Poster, Choice QR), або обмежуються лише переглядом меню (OddMenu). Система, що розробляється, покликана стати збалансованим інструментом, що поєднує автономність гостя (замовлення, оплата) із унікальними локальними запитами, як-от генерація PDF-меню та прямий виклик офіціанта. Це дозволить закладам малого

формату підвищити швидкість і ефективність обслуговування та мінімізувати витрати.

1.4 Концепція системи та об'єкт автоматизації

Об'єктом автоматизації є операційний цикл обслуговування гостя в закладі громадського харчування — від моменту його приходу до завершення оплати. В традиційній моделі цей цикл включає процеси очікування офіціанта, усне або письмове прийняття замовлення, його ручна передача на кухню, відстеження готовності страв персоналом, доставка замовлення та формування рахунку. Кожен із зазначених процесів є потенційним джерелом затримок, людських помилок і додаткового навантаження на персонал.

Система, що розробляється, автоматизує більшість із перелічених процесів, залишаючи людський фактор лише там, де він є необхідним — у безпосередній взаємодії з гостем та приготуванні страв.

В основі системи лежить модель BYOD (Bring Your Own Device), де смартфон гостя виступає персональним терміналом для замовлення без встановлення сторонніх застосунків. Клієнтський інтерфейс реалізований у форматі SPA (Single Page Application), що забезпечує миттєве завантаження, коректну роботу на мобільних пристроях різних платформ та можливість роботи за нестабільного з'єднання.

Водночас система не виключає класичної моделі обслуговування: офіціант може самостійно сформувати або відредагувати замовлення через власний інтерфейс. Така гібридна модель дозволяє закладу адаптувати систему під власний формат роботи без зміни усталених процесів обслуговування.

1.5 Рольова модель системи

Система обслуговує чотири типи користувачів із розмежованим доступом.

Гість є основним зовнішнім користувачем системи. Доступ до меню здійснюється через сканування QR-коду без обов'язкової реєстрації. Гість може переглядати меню з фільтрацією за категоріями, формувати кошик, відстежувати статус приготування замовлення в режимі реального часу, доповнювати замовлення після його відправлення, викликати офіціанта через інтерфейс та здійснювати онлайн-оплату з можливістю розділення рахунку. Авторизація через Google-акаунт надає доступ до збереженої історії замовлень та системи відгуків.

Кухар працює з системою через інтерфейс KDS (Kitchen Display System) — спеціалізований дисплей на кухні, що отримує нові замовлення миттєво через WebSocket-з'єднання. Кухар керує станами приготування кожної позиції у замовленні.

Офіціант має доступ до мобільної панелі персоналу з інтерактивною картою залу, що відображає поточний стан кожного столика: вільний, зайнятий або очікує на рахунок. Офіціант може редагувати активні замовлення за зверненням гостя та отримує сповіщення про виклик.

Адміністратор має повний доступ до всіх модулів системи: управління персоналом та рівнями доступу, конструктор меню з управлінням цінами та категоріями, модуль аналітики, генератор PDF-меню, налаштування QR-кодів для столиків та налаштування параметрів ресторану.

1.6 Функціональні модулі системи

Модуль QR-менеджменту забезпечує генерацію унікальних QR-кодів для кожного столика з прив'язкою до його фізичного розташування в залі. Для підвищення гнучкості реалізовано динамічний QR з проміжним редиректом, що дозволяє змінювати цільову URL-адресу без перевипуску коду. Після сканування система автоматично ідентифікує столик і прив'язує до нього всі подальші дії гостя в межах сесії, а механізм Smart Session Recovery відновлює перервану сесію при повторному скануванні або перезавантаженні сторінки.

Модуль замовлень у реальному часі реалізує передачу підтверджених замовлень від клієнтського інтерфейсу до кухонного дисплея через WebSocket-з'єднання без перезавантаження сторінки. Система підтримує одне активне замовлення на столик: повторне сканування QR-коду надає гостю доступ до перегляду меню без можливості розміщення нового замовлення. Статус замовлення оновлюється одночасно на кухонному дисплеї та в інтерфейсі гостя, а функція груп подачі дозволяє гостю об'єднувати страви в блоки для їх одночасного приготування та подачі.

Модуль кухонного дисплея (KDS) функціонує у двох режимах відображення: Order View, що показує замовлення як єдину картку, та Table View, що групує всі активні позиції по столиках. Персонал кухні послідовно переводить страви між статусами, а зміна статусу миттєво відображається в інтерфейсі гостя через WebSocket.

Модуль виклику персоналу підтримує два сценарії взаємодії гостя з офіціантом. Загальний виклик доступний у будь-який момент сесії для вирішення довільних питань. Виклик для готівкової оплати стає доступним лише після того, як усі страви замовлення отримали статус поданих, що унеможливорює передчасне закриття рахунку.

Фінансовий модуль підтримує як сповіщення для персоналу про оплату готівкою, так і повноцінну онлайн-оплату через інтегрований платіжний шлюз LiqPay. Онлайн-оплата, аналогічно до готівкової, стає доступною лише після подачі всіх страв. Операції скасування замовлення (Cancelled) є інструментами офіціанта або адміністратора і недоступні гостю.

Модуль меню забезпечує повноцінне управління структурою пропозиції закладу через CRUD-операції над стравами, категоріями, інгредієнтами та додатками (add-ons). Гість може переглядати детальний склад кожної позиції, а наявність модифікаторів і add-ons дозволяє персоналізувати замовлення без залучення персоналу.

Модуль аналітики та звітності надає адміністратору дашборд із показниками продажів, середнього чека та популярності окремих позицій меню у вигляді графіків і зведених таблиць за обраний період.

Генератор PDF-меню автоматично формує стилізований друкований документ на основі актуальних даних із бази даних, що призначений для фізичного друку та розміщення в залі.

Модуль авторизації реалізує повноцінну систему автентифікації, що підтримує два методи входу: за допомогою електронної пошти та пароля, а також через Google OAuth 2.0. Модуль також охоплює управління персоналом та процедуру створення нового ресторану.

Система відгуків дозволяє авторизованим гостям залишати оцінки окремим стравам та закладу в цілому після завершення замовлення.

1.7 Позиціонування на ринку

Система позиціонується як спеціалізоване SaaS-рішення (Software as a Service) для закладів харчування малого та середнього формату. Модель передбачає надання повного функціоналу платформи як хмарної послуги: заклад не купує програмне забезпечення і не розгортає власну інфраструктуру — натомість отримує доступ до готової системи без залучення технічних спеціалістів для впровадження.

Ключова відмінність від конкурентів полягає у цільовому фокусі: на відміну від комплексних POS-платформ, система не претендує на повну автоматизацію всіх бізнес-процесів закладу, а вирішує конкретну задачу — забезпечити повний цикл взаємодії гостя із закладом від сканування QR-коду до завершення оплати. Порівняно з вузькоспеціалізованими сервісами цифрового меню система надає значно ширший операційний функціонал за зіставну вартість.

Унікальною конкурентною перевагою системи є розширений real-time функціонал, що забезпечує миттєвий двосторонній зв'язок між гостем та персоналом. На відміну від статичних аналогів, система дозволяє клієнту викликати

офіціанта одним натисканням у веб-інтерфейсі та здійснювати редагування або доповнення замовлення «на ходу» без залучення персоналу. Додатковою перевагою є гібридна модель обслуговування, що поєднує самостійне замовлення гостем і класичну роботу офіціанта, дозволяючи закладу адаптувати систему під власні потреби без повної зміни бізнес-процесів.

1.8 Модель монетизації

Система базується на моделі Freemium із поділом функціоналу на два тарифні плани, що дозволяє закладам розпочати цифровізацію без початкових витрат і переходити на платний тариф у міру зростання потреб.

Базовий тариф (Free) надається без обмежень у часі та включає мінімально необхідний для роботи закладу функціонал:

- повний цикл прийому та обслуговування замовлень: QR-меню, самостійне замовлення гостем, кухонний дисплей (KDS), панель офіціанта з картою залу;
- виклик офіціанта через інтерфейс, включаючи виклик для готівкової оплати (cash_payment);
- оплата виключно готівкою через підтвердження офіціантом;
- управління меню з обмеженням: до 5 категорій, до 50 страв, до 3 зображень на страву;
- до 3 акаунтів персоналу сумарно (включно з адміністратором);
- базові налаштування ресторану: назва, адреса, тип кухні, контактні дані, мови інтерфейсу;
- система відгуків вимкнена: гості не мають доступу до форми відгуку.

Розширений тариф (Premium) орієнтований на заклади, які потребують повного функціоналу платформи та інструментів для збільшення прибутку:

- зняття обмежень на меню (необмежена кількість категорій, страв та зображень);

- інтеграція платіжного шлюзу LiqPay для безготівкової оплати онлайн через карту або Apple/Google Pay;
- багаторольовий персонал: підтримка ролей waiter, cook та комбінованої ролі waiter_cook, необмежена кількість акаунтів;
- повноцінний аналітичний модуль: виручка, середній чек, конверсія оплат, час приготування, топ страв, статистика відгуків за весь період;
- модуль управління відгуками для адміністратора (перегляд, фільтрація, видалення);
- активація системи залишення відгуків з боку гостей;
- автоматичний генератор PDF-меню для фізичного друку;
- інтеграція з Google Translate для автоматичного перекладу всіх двомовних полів вводу (назв та описів страв, категорій), що знімає необхідність ручного перекладу при веденні меню.

Розподіл функціоналу реалізовано на рівні бази даних (поле plan у документі ресторану) та контролюється проміжним програмним забезпеченням (middleware) на серверній частині, а також умовним рендерингом елементів інтерфейсу на клієнтській частині.

1.9 Обґрунтування вибору технологічного стеку

Вибір технологій для реалізації системи обумовлений вимогами до продуктивності, масштабованості та швидкості розробки.

ReactJS обрано для реалізації клієнтської частини з огляду на його компонентну архітектуру, що дозволяє незалежно розробляти та повторно використовувати елементи інтерфейсу як для мобільної версії гостя, так і для адміністративного дашборду. Віртуальний DOM забезпечує високу швидкість оновлення інтерфейсу при частих змінах стану — що важливо для відображення статусів замовлень у режимі реального часу. Декларативний підхід до побудови інтерфейсу та однонаправлений потік даних, покладені в основу React, спрощують

міркування про стан застосунку та зменшують кількість помилок при масштабуванні кодової бази [9].

Node.js обрано для серверної частини завдяки його асинхронній неблокуючій архітектурі, що є оптимальною для обробки великої кількості одночасних з'єднань — зокрема WebSocket-сесій від кількох столиків одночасно. Використання JavaScript як єдиної мови для клієнтської та серверної частин спрощує розробку та підтримку кодової бази [9].

MongoDB обрано як базу даних завдяки документно-орієнтованій моделі зберігання даних, що природно відповідає структурі меню ресторану: вкладені категорії, варіації страв і модифікатори зручно представляються у форматі JSON-документів без необхідності складних реляційних зв'язків. Гнучка схема дозволяє оперативно змінювати структуру меню без міграцій бази даних [10].

Google API використовується для реалізації автентифікації гостей за схемою OAuth 2.0, що забезпечує максимально швидкий вхід без необхідності повільної реєстрації акаунта та підвищує довіру користувачів завдяки знайомому та швидкому механізму авторизації.

2 ФОРМУВАННЯ ВИМОГ

2.1 Огляд функціональних вимог

Функціональні вимоги системи розподілені на п'ятнадцять логічних модулів, кожен з яких відповідає окремому бізнес-сценарію. Кожна вимога має унікальний ідентифікатор формату SYS-<МОДУЛЬ>-<номер>, а кожен критерій приймання — ідентифікатор формату AC-<МОДУЛЬ>-<номер> та сформульований за схемою Given/When/Then. Нижче наведено повний опис кожного модуля системи, описаного у специфікації вимог до програмного забезпечення (SRS).

2.1.1 Модуль QR-менеджменту та ідентифікація столика

Модуль забезпечує ідентифікацію столика через унікальний QR-код та керування життєвим циклом гостьової сесії. Ключовою особливістю є механізм Smart Session Recovery, що відновлює попередню сесію при повторному скануванні замість створення нової, а також гарантія одного активного замовлення на столик. Пріоритет: критичний.

Функціональні вимоги модуля:

- SYS-QR-01: Short-code формату /qr/{shortCode}; 4–8 символів [A-Z0-9]; криптографічно безпечна генерація (crypto.randomBytes).
- SYS-QR-02: Сервер при GET /qr/{shortCode} перевіряє cookies на активний session token для поточного tableId.
- SYS-QR-03: Прив'язка short-code → tableId зберігається в БД та може змінюватись адміном.
- SYS-QR-04: Адмін може тимчасово деактивувати столик без зміни фізичного QR.
- SYS-QR-05: Session token має термін дії 24 години (Max-Age: 86400); після закінчення генерується нова сесія.
- SYS-QR-06: Session token зберігається в HTTP cookie з атрибутами HttpOnly, Secure, SameSite=Strict.

- SYS-QR-07: Адмін завантажує QR-зображення у форматі PNG або PDF для одноразового друку.
 - SYS-QR-08: Карта залу в панелі адміна відображає поточний стан кожного столика.
 - SYS-QR-09: Якщо session token активний — сервер повертає існуючу сесію (кошик, статус замовлення) замість створення нової (Smart Session Recovery).
 - SYS-QR-10: Нова сесія створюється лише якщо: немає токена в cookies / token прострочений / token анульований / token належить іншому tableId.
 - SYS-QR-11: При переведенні столика в статус «Вільний» система інвалідує всі активні session token цього tableId автоматично.
 - SYS-QR-12: При GET /qr/{shortCode} для неіснуючого shortCode сервер повертає HTTP 404 з інформаційною сторінкою.
 - SYS-QR-13: При скануванні QR столика з активним замовленням — відповідь містить прапорець tableHasActiveOrder: true; новий Order не створюється.
- Критерії приймання модуля:
- AC-QR-01: Given shortCode існує та активний, When гість сканує QR вперше, Then HTTP 302 до /menu з новим sessionToken за ≤ 500 мс.
 - AC-QR-02: Given гість має активний token для tableId T1, When повторно сканує QR, Then повертається та сама сесія; новий Order не створюється.
 - AC-QR-03: Given shortCode не існує, When GET /qr/{unknownCode}, Then HTTP 404 «QR-код не знайдено».
 - AC-QR-04: Given столик деактивований, When гість сканує QR, Then «Столик тимчасово недоступний»; Session та Order не створюються.
 - AC-QR-05: Given столик $\rightarrow$ «Вільний», Then усі session token цього tableId анулюються; наступне сканування гарантовано створює нову сесію.
 - AC-QR-06: Given за столиком вже є активне замовлення, When другий гість сканує QR, Then відповідь містить tableHasActiveOrder: true; гість бачить меню без кнопки «Підтвердити замовлення».

2.1.2 Модуль меню: інгредієнти, модифікатори та добавки

Модуль реалізує відображення меню з підтримкою гнучкої кастомізації страв через знімні інгредієнти, добавки (add-ons) та обов'язкові варіанти вибору (ComponentGroups). Передбачено клієнтський пошук на основі кешованих даних для миттєвої реакції інтерфейсу. Пріоритет: високий.

Функціональні вимоги модуля:

- SYS-MENU-01: Кожна страва повинна мати список інгредієнтів, що керується адміном.
- SYS-MENU-02: Адмін позначає кожен інгредієнт як «можна прибрати» або «обов'язковий».
- SYS-MENU-03: Гість може прибрати лише інгредієнти, позначені адміном як дозволені для виключення.
- SYS-MENU-04: Кожна страва може мати перелік add-ons; кожен add-on має назву та ціну (0 або більше).
- SYS-MENU-05: Гість може обрати один або кілька add-ons до страви.
- SYS-MENU-06: Кожна страва може мати перелік ComponentGroups, в якій клієнт обов'язково має обрати одну.
- SYS-MENU-07: Гість може залишити довільний текстовий коментар до кожної окремої позиції (максимум 300 символів), валідація на клієнті та сервері.
- SYS-MENU-08: Виключені інгредієнти, add-ons, ComponentGroups та коментар передаються на KDS разом із позицією.
- SYS-MENU-09: Позиції зі стоп-листа відображаються як недоступні при завантаженні та перевіряються перед створенням замовлення.
- SYS-MENU-10: При недоступності фото відображається placeholder без порушення відображення решти даних страви.
- SYS-MENU-11: Пошук за назвою страви доступний глобально по всьому меню та локально в межах категорії; фільтрація case-insensitive з частковим

співпадінням; реалізується на клієнті без запиту до сервера на основі кешованих даних меню.

– SYS-MENU-12: Поле пошуку присутнє на головній сторінці меню (глобальний) та на сторінці кожної категорії (локальний).

– SYS-MENU-13: Результати пошуку оновлюються при кожному введеному символі.

Критерії приймання модуля:

– AC-MENU-01: Given страва має 5 інгредієнтів (3 знімних, 2 обов'язкових), Then знімні мають чекбокс, обов'язкові — ні.

– AC-MENU-02: Given страва має ComponentGroup «Гарнір», Then кнопка «Додати до кошика» неактивна до вибору варіанту.

– AC-MENU-03: Given гість виключив «цибулю» та підтвердив замовлення, Then на KDS відображається «Без цибулі».

– AC-MENU-04: Given add-on 20 грн, When гість обирає, Then ціна позиції збільшується на 20 грн в реальному часі.

– AC-MENU-05: Given гість вводить 301+ символ у коментар, Then символи понад 300 не вводяться; лічильник показує «300/300».

– AC-MENU-06: Given кухар додав страву до стоп-листа, Then при спробі замовлення позиція недоступна.

– AC-MENU-07: Given меню завантажено, When гість вводить «піца» у глобальний пошук, Then відображаються всі страви з «піца» у назві (case-insensitive) без запиту до сервера.

– AC-MENU-08: Given гість перебуває в категорії «Піца» та вводить «маргарит» у локальний пошук, Then відображаються лише страви цієї категорії, що містять «маргарит» у назві.

– AC-MENU-09: Given пошуковий запит не знайшов жодної страви, Then відображається «Страв не знайдено».

2.1.3 Модуль формування замовлення: кошик та групи подачі

Гість формує кошик зі стравами та може об'єднати їх у групи подачі — набори страв для одночасного приготування та подачі. На кухонному дисплеї кожна група відображається як монолітний блок зі спільним статусом, що дозволяє синхронізувати подачу складних замовлень.

Функціональні вимоги модуля:

- SYS-CART-01: Гість може об'єднувати страви в іменовані групи подачі в межах одного замовлення.
- SYS-CART-02: Страви без явного групування потрапляють до «Основної групи» за замовчуванням.
- SYS-CART-03: Кожна новостворена група автоматично має назву «Група подачі ...» з відповідним номером.
- SYS-CART-04: На KDS страви — монолітні блоки за групами.
- SYS-CART-05: Статус групи є спільним для всіх страв у ній; зміна статусу групи змінює статус кожної страви одночасно.
- SYS-CART-06: Гість отримує WebSocket-сповіщення коли кожна з його груп змінює статус.
- SYS-CART-07: Гість може змінювати кількість / видаляти позиції до підтвердження.
- SYS-CART-08: При підтвердженні сервер виконує фінальну валідацію на стоп-лист; за наявності недоступних позицій — HTTP 422 зі списком.
- SYS-CART-09: Кнопка «Підтвердити замовлення» неактивна для порожнього кошика.
- SYS-CART-10: При помилці сервера під час підтвердження кошик зберігається на клієнті.

Критерії приймання модуля:

- AC-CART-01: Given гість додав 3 страви, When натискає «Підтвердити замовлення», Then замовлення з'являється на KDS за ≤ 300 мс.

– AC-CART-02: Given гість створив групи «Стартери» та «Основне», When замовлення підтверджено, Then на KDS відображаються 2 окремі блоки з відповідними назвами.

– AC-CART-03: Given одна страва потрапила до стоп-листа, When гість натискає «Підтвердити», Then замовлення не відправляється; відображається список недоступних страв з кнопкою «Видалити недоступні».

– AC-CART-04: Given кошик порожній, When відображається сторінка кошика, Then кнопка «Підтвердити замовлення» має атрибут disabled.

– AC-CART-05: Given сервер повернув HTTP 500 при підтвердженні, When гість бачить помилку, Then кошик незмінний; повторна спроба можлива.

2.1.4 Модуль управління замовленням столику

За кожним столиком може існувати щонайбільше одне активне замовлення (зі статусом open) одночасно. При спробі другого гостя розмістити замовлення за тим самим столиком система повертає HTTP 409 (TABLE_OCCUPIED). Другий гість, що сканує QR столика, отримує меню в режимі «тільки перегляд» — він може переглядати страви, але не може додавати позиції до кошика або підтверджувати замовлення. Пріоритет: критичний.

Функціональні вимоги модуля:

– SYS-ORDER-01: Якщо декілька людей сканує QR, то лише перший, хто зробить замовлення, буде мати доступ до нього; всі інші зможуть лише переглядати меню.

– SYS-ORDER-02: Сервер перевіряє наявність активного замовлення за tableId перед створенням нового; якщо є — HTTP 409 TABLE_OCCUPIED та tableHasActiveOrder: true.

– SYS-ORDER-03: Відповідь QR-ендпоінту для столика з активним замовленням містить tableHasActiveOrder: true.

- SYS-ORDER-04: Гість, який отримав `tableHasActiveOrder: true`, бачить меню без можливості розмістити замовлення (режим «тільки перегляд»).
- SYS-ORDER-05: Гість бачить лише власне замовлення; доступ до чужого → HTTP 403.
- SYS-ORDER-06: Столик → `free`, коли замовлення переходить у фінальний статус (`completed_cash` / `completed_epay` / `cancelled`) — автоматично.
- SYS-ORDER-07: Офіціант може вручну закрити столик незалежно від стану замовлення; залишкове активне замовлення → `cancelled` з причиною «Столик закрито вручну».
- SYS-ORDER-08: При вході авторизованого гостя — система перевіряє наявність активного замовлення та відновлює стан замовлення та `restaurantId` автоматично.

Критерії приймання модуля:

- AC-ORDER-01: Given за столиком є активне замовлення (`open`), When другий гість сканує QR, Then він бачить меню з неактивним кошиком; кнопка «Підтвердити замовлення» неактивна.
- AC-ORDER-02: Given замовлення → `completed_epay`, Then столик → «Вільний» за ≤ 300 мс.
- AC-ORDER-03: Given двоє ініціюють POST /orders одночасно, Then другий → HTTP 409; лише одне замовлення створено.
- AC-ORDER-04: Given авторизований гість входить до акаунта і має активне замовлення, Then його замовлення `restaurantId` відновлюється автоматично; гість перенаправляється до меню.
- AC-ORDER-05: Given неавторизований гість, який має активне замовлення, втрачає до нього доступ (очищення cookie або зміна пристрою), Then після звернення до персоналу офіціант може ввімкнути режим відновлення і при повторному скануванні QR сесія з даними про замовлення буде відновлена.

2.1.5 Модуль передачі замовлення на кухню (KDS)

Модуль забезпечує миттєву передачу підтверджених замовлень на кухонний дисплей через WebSocket. Статуси страв змінюються однонаправлено за моделлю `waiting` → `cooking` → `ready` → `served` з короткочасною можливістю відкату. Пріоритет: критичний.

Функціональні вимоги модуля:

- SYS-KDS-01: Нові замовлення на KDS без ручного оновлення (WebSocket `ORDER_NEW`), затримка ≤ 300 мс.
- SYS-KDS-02: Картка замовлення містить: номер столика, `orderId`, групи подачі зі стравами, виключені інгредієнти, `add-ons`, коментарі та час надходження.
- SYS-KDS-03: Групи подачі відображаються як монолітні блоки з єдиною кнопкою зміни статусу.
- SYS-KDS-04: Статуси групи: Очікує → Готується → Готово (однаправлений перехід; зниження заборонено).
- SYS-KDS-05: Кухар може додавати/знімати позиції зі стоп-листа.
- SYS-KDS-06: Зміна стоп-листа відображається в меню всіх клієнтів та блокує замовлення з ними.
- SYS-KDS-07: KDS підтримує два режими: Order View (кожне замовлення — окрема картка) та Table View (всі замовлення столика — єдиний блок).
- SYS-KDS-08: При розриві WebSocket KDS відображає індикатор та автоматично відновлює з'єднання.
- SYS-KDS-09: Після відновлення WebSocket сервер надсилає `event replay` пропущених подій від останнього підтвердженого `event_id`.
- SYS-KDS-10: Спроба зниження статусу групи може бути виконана лише в першу хвилину після минулої зміни; в інших випадках відхиляється сервером (HTTP 400).

Критерії приймання модуля:

- AC-KDS-01: Given замовлення з 2 групами, Then 2 блоки на KDS, які мають сталий порядок за ≤ 300 мс.
- AC-KDS-02: Given WebSocket KDS розірваний і відновлений, Then актуальний стан замовлень без ручного оновлення за ≤ 5 с.
- AC-KDS-03: Given кухар позначив групу «ready», Then GROUP_STATUS_UPDATED $\rightarrow$ пристрій гостя за ≤ 300 мс.
- AC-KDS-04: Given кухар помилково змінює статус групи до «ready», When кухар намагається повернути «cooking», Then успіх, статус повертається, клієнт сповіщений про цю помилку.
- AC-KDS-05: Given група «ready», When кухар намагається $\rightarrow$ «cooking», Then HTTP 400; статус не змінюється.

2.1.6 Модуль виклику офіціанта

Система підтримує два типи виклику офіціанта: загальний виклик (call) — гість може викликати офіціанта у будь-який момент після підтвердження замовлення, та виклик для готівкової оплати (cash_payment). Офіціант отримує WebSocket-сповіщення за ≤ 300 мс. Непідтверджені виклики старші 3 хвилин візуально виділяються. Повторний виклик, поки є вже активний, ігнорується. Пріоритет: високий.

Функціональні вимоги модуля:

- SYS-CALL-01: Кнопка загального виклику доступна в будь-який момент після підтвердження замовлення.
- SYS-CALL-02: Кнопка виклику для готівкової оплати (cash_payment) є активною протягом усього замовлення та при натисканні закриває зміну замовлення.
- SYS-CALL-03: WebSocket-сповіщення на панель офіціанта за ≤ 300 мс для обох типів.
- SYS-CALL-04: Офіціант підтверджує виклик одним натисканням.

- SYS-CALL-05: Сповідання містить: номер столика, orderId, тип виклику, час виклику.
- SYS-CALL-06: Виклики сортуються за їх часом, де перший найдовший.
- SYS-CALL-07: Повторний виклик від того самого гостя протягом 60 секунд ігнорується; гість отримує повідомлення.
- SYS-CALL-08: Виклик, що надійшов під час відключення WebSocket офіціанта, доставляється після відновлення з'єднання (event replay).

Критерії приймання модуля:

- AC-CALL-01: Given гість натискає «Викликати офіціанта», Then на панелі офіціанта з'являється сповіщення за ≤ 300 мс.
- AC-CALL-02: Given гість натиснув виклик, When повторно протягом 60 с, Then новий WaiterCall не створюється.
- AC-CALL-03: Given гість натискає «Оплатити готівкою», Then офіціант отримує WAITER_CALL_CASH за ≤ 300 мс.
- AC-CALL-04: Given офіціант підтверджує cash_payment виклик, Then замовлення $\rightarrow$ completed_cash; столик $\rightarrow$ free; WebSocket PAYMENT_COMPLETED за ≤ 300 мс.

2.1.7 Модуль відстеження статусу замовлення

Гість бачить стан кожної страви в реальному часі через WebSocket. Статус замовлення (open / cancelled / completed_cash / completed_pay) не є похідним від статусів страв і відображає фінансово-операційний стан. Редагування заблоковане для страв зі статусом cooking+; після переходу замовлення у фінальний статус (cancelled / completed_*) — будь-які зміни неможливі. Пріоритет: високий.

Функціональні вимоги модуля:

- SYS-STAT-01: Інтерфейс відображає статус по кожній групі подачі: Прийнято / Готується / Готово.

- SYS-STAT-02: Оновлення статусу через WebSocket без перезавантаження сторінки.
- SYS-STAT-03: Гість може доповнити замовлення новими позиціями (лише коли `Order.status = open`).
- SYS-STAT-04: Гість НЕ може видаляти або зменшувати кількість позицій у групі зі статусом «Готується» або «Готово».
- SYS-STAT-05: Редагування (видалення/зменшення кількості) дозволене лише для позицій у групах зі статусом «Очікує».
- SYS-STAT-06: Після `cancelled` або `completed_*` — будь-які зміни → HTTP 409.
- SYS-STAT-07: При розриві WebSocket клієнт відображає індикатор втрати з'єднання і відновлює його автоматично (≤ 5 с).

Критерії приймання модуля:

- AC-STAT-01: Given кухар змінив статус групи, When подія надходить на пристрій гостя, Then статус оновлюється без перезавантаження за ≤ 300 мс.
- AC-STAT-02: Given група має статус «Готується», When гість намагається видалити позицію, Then відображається «Страву вже готують — зверніться до офіціанта»; позиція залишається.
- AC-STAT-03: Given `Order.status = completed_епay`, When спроба зміни, Then HTTP 409.

2.1.8 Модуль оплати

Система використовує модель пост-пей. Платіжний шлюз: LiqPay. Гість оплачує замовлення через LiqPay або готівкою. Оплата може бути здійснена і до закінчення замовлення, але в такому разі подальше редагування замовлення заблоковане. Пріоритет: критичний.

Функціональні вимоги модуля:

- SYS-PAY-01: Кнопка онлайн-оплати (LiqPay) може бути натиснута при стані замовлення open незалежно від стану страв у ньому.
- SYS-PAY-02: Карткові дані не зберігаються на серверах системи — обробляються виключно LiqPay.
- SYS-PAY-03: Після закриття замовлення (completed_cash / completed_epay / cancelled) → столик «Вільний».
- SYS-PAY-04: Всі транзакції (CASH, Cancelled, walk-out) зберігаються в БД та незмінному audit log.
- SYS-PAY-05: Готівкова оплата підтверджується офіціантом через WaiterCall типу cash_payment або вручну офіціантом; після підтвердження Order.status → completed_cash.
- SYS-PAY-06: Після completed_* або cancelled — будь-які зміни замовлення → HTTP 409.
- SYS-PAY-07: При LiqPay timeout Order.status без змін; інформативне повідомлення користувачу.
- SYS-PAY-08: Fallback: повторні запити статусу до LiqPay через 1/5/15 хв при ненадходженні webhook.

Критерії приймання модуля:

- AC-PAY-01: Given LiqPay підтверджує, When webhook надходить, Then Order.status → completed_epay і підтвердження для гостя за ≤ 3 с.
- AC-PAY-02: Given LiqPay не відповідає > 30 с, Then повідомлення про помилку; Order.status = open.

2.1.9 Модуль аналітики та звітності

Адміністратор має доступ до аналітичного дашборду для перегляду бізнес-метрик. Дані формуються з транзакцій MongoDB у реальному часі. Пріоритет: середній (розширений тариф).

Функціональні вимоги модуля:

- SYS-ANLT-01: Загальна виручка за обраний період.
- SYS-ANLT-02: Середній чек замовлення за обраний період.
- SYS-ANLT-03: Топ-10 найпопулярніших страв за кількістю замовлень.
- SYS-ANLT-04: Вибір довільного діапазону дат (максимум 365 днів за один запит).
- SYS-ANLT-05: Дані формуються з транзакцій MongoDB.
- SYS-ANLT-06: Кількість та відсоток cancelled випадків за обраний період.
- SYS-ANLT-07: Середній час приготування замовлення (від cooking до ready) по стравах та категоріях.
- SYS-ANLT-08: Конверсія оплат: відсоток PAID проти cancelled.
- SYS-ANLT-09: Для діапазонів > 365 днів — пропозиція вивантаження CSV.

Критерії приймання модуля:

- AC-ANLT-01: Given адмін обирає діапазон до 90 днів, When запит виконується, Then дашборд завантажується за ≤ 3 с.
- AC-ANLT-02: Given транзакція з типом «cancelled», When вона включена в аналітику, Then вона входить до статистики cancelled, але НЕ до загальної виручки.

2.1.10 Модуль генератора PDF-меню

Адміністратор формує PDF-документ із поточним меню для фізичного друку.
Пріоритет: середній (розширений тариф).

Функціональні вимоги модуля:

- SYS-PDF-01: PDF формується на основі поточних даних меню з MongoDB.
- SYS-PDF-02: PDF містить: назву закладу, категорії, назви страв, описи, інгредієнти та ціни.
- SYS-PDF-03: Платний тариф — вибір між шаблонами оформлення.
- SYS-PDF-04: Формат A4, придатний для якісного друку.
- SYS-PDF-05: При помилці генерації адмін отримує повідомлення «Не вдалось згенерувати PDF. Спробуйте пізніше».

Критерії приймання модуля:

- AC-PDF-01: Given меню містить 100 позицій, When адмін ініціює генерацію, Then PDF файл формується за ≤ 5 с і доступний для завантаження.
- AC-PDF-02: Given PDF-рушій недоступний, When адмін ініціює генерацію, Then відображається помилка; інші функції системи залишаються доступними.

2.1.11 Модуль системи відгуків

Авторизовані гості залишають відгуки: про ресторан (один на замовлення) та по одному на кожну замовлену страву. Доступно лише після `Order.status = completed_cash` або `completed_pay`. Пріоритет: середній (розширений тариф).

Функціональні вимоги модуля:

- SYS-REV-01: Відгуки доступні лише авторизованим гостям для замовлень зі статусом `completed_cash` або `completed_pay`.
- SYS-REV-02: `RestaurantReview` — один на замовлення від одного акаунта.
- SYS-REV-03: `DishReview` — один на кожну `OrderItem` від одного акаунта.
- SYS-REV-04: Обидва типи: оцінка 1–5 та необов'язковий коментар (≤ 500 символів).
- SYS-REV-05: Адмін може переглядати та видаляти відгуки з фільтрацією за датою, стравою, рейтингом.

Критерії приймання модуля:

- AC-REV-01: Given авторизований гість з `completed_pay` замовленням, When залишає `RestaurantReview` з оцінкою 4, Then відгук у панелі адміна.
- AC-REV-02: Given вже є `RestaurantReview`, When повторна спроба, Then HTTP 409.
- AC-REV-03: Given анонімний гість, When відкриває форму відгуку, Then HTTP 401.
- AC-REV-04: Given замовлення не `completed_*`, Then HTTP 403 «Відгуки доступні лише для завершених замовлень».

2.1.12 Модуль авторизації та облікових записів

Модуль підтримує два паралельні методи входу: email+пароль із bcrypt-хешуванням та Google OAuth 2.0. Для гостей автентифікація опціональна — система підтримує анонімне користування з прив'язкою сесії до QR-сканування. Пріоритет: критичний.

Функціональні вимоги модуля:

- SYS-AUTH-01: Система підтримує реєстрацію та вхід через email+пароль для всіх типів акаунтів.
- SYS-AUTH-02: Система підтримує вхід через Google OAuth 2.0 для гостей та персоналу.
- SYS-AUTH-03: Для персоналу (кухар, офіціант, адмін) наявність email+пароля є обов'язковою умовою.
- SYS-AUTH-04: Гість може замовляти анонімно без будь-якої авторизації.
- SYS-AUTH-05: При вході нового користувача через Google — система автоматично створює акаунт; ім'я з Google-профілю.
- SYS-AUTH-06: При повторному вході через Google — система знаходить існуючий акаунт за Google ID.
- SYS-AUTH-07: Ім'я в профілі можна змінити незалежно від методу створення акаунта.
- SYS-AUTH-08: Гість і персонал можуть прив'язати Google-акаунт до існуючого email-акаунта.
- SYS-AUTH-09: Відв'язати Google можна лише за наявності альтернативного методу входу.
- SYS-AUTH-10: Паролі зберігаються виключно у вигляді bcrypt-хешів (salt-rounds ≥ 12).
- SYS-AUTH-11: JWT access-токен діє ≤ 1 годину; refresh-токен зберігається зашифровано.

- SYS-AUTH-12: API перевіряє роль користувача (RBAC) для кожного захищеного ендпоінта.
- SYS-AUTH-13: При недоступності Google OAuth система продовжує роботу через email+пароль.
- SYS-AUTH-14: При вході авторизованого гостя — система перевіряє наявність активного замовлення та відновлює restaurantId автоматично.
- SYS-AUTH-15: Пряма реєстрація (email+пароль) активується одразу без email-верифікації; для onboarding нового ресторану обов'язкова email-верифікація (/onboarding/check-email → /onboarding/confirm).
- SYS-AUTH-16: Кожен гість може за бажанням видалити свій акаунт.

Критерії приймання модуля:

- AC-AUTH-01: Given нова реєстрація, Then пароль у БД — bcrypt-хеш; відкритий текст відсутній; акаунт одразу активний.
- AC-AUTH-02: Given Google OAuth недоступний, When гість натискає «Увійти через Google», Then відображається помилка; кнопка email-входу залишається функціональною.
- AC-AUTH-03: Given персонал роллю «cook», When GET /api/admin/analytics, Then HTTP 403.
- AC-AUTH-04: Given гість увійшов виключно через Google (без пароля), When намагається відв'язати Google, Then HTTP 400 «Встановіть пароль перед відв'язуванням Google».

2.1.13 Модуль управління меню

Модуль надає адміністратору повний CRUD-функціонал для всіх сутностей меню: категорій, страв, інгредієнтів, додатків та груп компонентів. Пріоритет: критичний.

Функціональні вимоги модуля:

- SYS-MGMT-01: Адмін може створювати, редагувати та видаляти (soft-delete) категорії.
- SYS-MGMT-02: Адмін може створювати, редагувати та видаляти (soft-delete) страви.
- SYS-MGMT-03: unitPrice вже розміщених OrderItem залишається незмінним (snapshot ціни).
- SYS-MGMT-04: Адмін керує інгредієнтами страви (знімні / обов'язкові).
- SYS-MGMT-05: Адмін керує add-ons страви (назва, ціна).
- SYS-MGMT-06: Адмін керує ComponentGroups (обов'язковий вибір варіанту).
- SYS-MGMT-07: Адмін керує sortOrder категорій та страв.
- SYS-MGMT-08: Будь-яка зміна меню → WebSocket MENU_UPDATED всім активним клієнтам.
- SYS-MGMT-09: Видалення категорії зі стравами → HTTP 409.
- SYS-MGMT-10: Зображення страв: JPG/PNG/WebP, ≤ 5 МБ.

Критерії приймання модуля:

- AC-MGMT-01: Given адмін додає страву за 150 грн, When гість відкриває меню, Then страва відображається за ≤ 500 мс після збереження.
- AC-MGMT-02: Given адмін змінює ціну з 100 на 120 грн, When гість вже має її в підтвердженому замовленні, Then unitPrice в OrderItem залишається 100 грн.
- AC-MGMT-03: Given адмін видаляє страву (soft-delete), Then MENU_UPDATED → клієнти; страва → «Недоступно».
- AC-MGMT-04: Given адмін видаляє категорію зі стравами, Then HTTP 409; категорія не видаляється.

2.1.14 Модуль управління персоналом та onboarding ресторану

Адміністратор керує обліковими записами персоналу. Перший адміністратор реєструється через процедуру Onboarding при створенні закладу в системі. Один

обліковий запис адміністратора прив'язаний рівно до одного ресторану. Email-повідомлення для запрошення персоналу не надсилаються — тимчасові паролі відображаються адміністратору в інтерфейсі одноразово. Для onboarding нового ресторану надсилається email-підтвердження. Пріоритет: високий.

Процедура реєстрації нового ресторану в системі складається з таких кроків:

- Крок 1: Власник / менеджер відкриває сторінку реєстрації та вводить: email, пароль, ім'я, назву та адресу ресторану.
- Крок 2: Система створює Restaurant (restaurantId) та обліковий запис першого адміністратора з роллю administrator.
- Крок 3: На email надсилається підтверджувальний лист; акаунт активується після підтвердження email.
- Крок 4: Адмін входить до системи та може розпочати налаштування: додавати столики, QR-коди, меню та персонал.
- Крок 5: Система генерує перший short-code та QR для тестового столика (опційно, «Quick Start»).

Функціональні вимоги модуля:

- SYS-STAFF-01: Перший адміністратор реєструється через Onboarding при створенні ресторану; акаунт активується після email-верифікації (маршрут /onboarding/check-email → /onboarding/confirm).
- SYS-STAFF-02: Адмін може створювати акаунти для кухарів та офіціантів (email + тимчасовий пароль).
- SYS-STAFF-03: При створенні акаунта система надсилає тимчасовий пароль співробітнику на вказаний email.
- SYS-STAFF-04: Адмін може змінювати роль: cook ↔ waiter ↔ administrator.
- SYS-STAFF-05: При зміні ролі всі активні JWT токени анулюються.
- SYS-STAFF-06: Адмін може деактивувати акаунт; всі активні сесії анулюються.

- SYS-STAFF-07: Ресторан повинен мати мінімум одного активного адміністратора.
- SYS-STAFF-08: Адмін не може видалити власний акаунт.
- SYS-STAFF-09: Список персоналу з фільтрацією за роллю та статусом.
- SYS-STAFF-10: Один адміністраторський акаунт прив'язаний рівно до одного ресторану; спроба реєстрації другого ресторану з тим самим email → HTTP 409.
- SYS-STAFF-11: Адмін може скинути пароль будь-якого співробітника.

Критерії приймання модуля:

- AC-STAFF-01: Given адмін створює акаунт кухаря, Then система надсилає тимчасовий пароль на вказаний email; співробітник входить в акаунт з отриманим паролем.
- AC-STAFF-02: Given адмін деактивує офіціанта, When офіціант намагається увійти, Then HTTP 401 «Акаунт деактивовано».
- AC-STAFF-03: Given єдиний адміністратор, When деактивація, Then HTTP 409.
- AC-STAFF-04: Given власник реєструє ресторан через Onboarding, Then restaurantId та акаунт адміна створені; обліковий запис одразу активний.
- AC-STAFF-05: Given адмін намагається зареєструвати другий ресторан зі своїм email, Then HTTP 409 «Обліковий запис вже прив'язаний до ресторану».

2.1.15 Модуль офлайн-режиму клієнтського інтерфейсу (PWA)

Клієнтський інтерфейс реалізується як Progressive Web App (PWA) з підтримкою часткового офлайн-режиму через Service Worker. Весь функціонал, що не вимагає серверної взаємодії, повинен залишатися доступним при відсутності інтернет-з'єднання. Session token зберігається в HttpOnly cookie; кошик та черга замовлень — у localStorage. Пріоритет: середній.

Функціональні вимоги модуля:

- SYS-PWA-01: Service Worker кешує статичні ресурси (JS, CSS, шрифти, іконки) при першому завантаженні застосунку.
- SYS-PWA-02: Дані меню (категорії, страви, інгредієнти, add-ons, ComponentGroups) кешуються після першого успішного завантаження.
- SYS-PWA-03: Кошик зберігається в localStorage; відновлюється при повторному відкритті навіть без мережі.
- SYS-PWA-04: Якщо гість натиснув «Підтвердити» в офлайн — замовлення ставиться в чергу в localStorage та відправляється при відновленні з'єднання з нотифікацією «Замовлення відправляється...».
- SYS-PWA-05: Черга очікуючих запитів зберігається в localStorage; при закритті та повторному відкритті браузера черга не втрачається.
- SYS-PWA-06: Клієнт відображає banner «Немає з'єднання» при відсутності мережі; banner зникає при відновленні.
- SYS-PWA-07: Пошук у меню функціонує офлайн на основі кешованих даних.

Критерії приймання модуля:

- AC-PWA-01: Given меню завантажено, When гість вимикає мережу, Then меню, деталі страв, пошук та кошик залишаються доступними.
- AC-PWA-02: Given гість додав страви до кошика офлайн, When мережа відновлена, Then кошик збережений; гість може підтвердити замовлення.
- AC-PWA-03: Given гість натиснув «Підтвердити» офлайн, Then система показує «Замовлення відправляється...»; після відновлення мережі замовлення відправляється автоматично; гість отримує підтвердження.
- AC-PWA-04: Given мережа відсутня, Then відображається banner «Немає з'єднання»; при відновленні banner зникає; статус замовлення оновлюється автоматично за ≤ 5 с.

2.2 Огляд нефункціональних вимог

Нефункціональні вимоги визначають якісні характеристики системи та поділяються на шість категорій.

2.2.1 Вимоги до продуктивності

Час завантаження клієнтського інтерфейсу не повинен перевищувати 2 секунди при підключенні 4G. Затримка передачі замовлення через WebSocket — не більше 300 мс. Час відповіді REST API — не більше 500 мс для 95% запитів. Система розрахована на одночасну обробку до 100 активних WebSocket-з'єднань на один ресторан. Доступність системи (uptime) — не менше 99,5% за календарний місяць.

2.2.2 Вимоги до безпеки

Усі мережеві з'єднання здійснюються виключно через HTTPS та WSS. Автентифікація персоналу реалізована через JWT-токени з обмеженим терміном дії. Паролі зберігаються виключно у вигляді bcrypt-хешів з параметром salt-rounds ≥ 12 . Карткові дані обробляються виключно платіжним шлюзом LiqPay і не потрапляють на сервери системи. Усі API-ендпоінти захищені перевіркою ролі користувача за моделлю RBAC. Реалізовано захист від типових векторів атак: XSS, CSRF, NoSQL-ін'єкцій. Фінансові операції фіксуються в незмінному audit log.

2.2.3 Атрибути якості

Зручність (Usability): гість виконує перше замовлення без інструкцій за не більше ніж три дії. Надійність (Reliability): автоматичне відновлення WebSocket-з'єднання після розриву за не більше 5 секунд. Масштабованість (Scalability): архітектура передбачає горизонтальне масштабування для обслуговування багатьох ресторанів. Супроводжуваність (Maintainability): компонентна структура клієнтської частини та модульна архітектура серверної частини забезпечують незалежне оновлення модулів. Переносимість (Portability): коректне відображення

на пристроях з шириною екрана від 320 px до 1920 px. Відновлюваність (Recoverability): Smart Session Recovery дозволяє відновити сесію після закриття браузера без втрати кошика та поточного статусу замовлення.

2.2.4 Вимоги до сумісності

Підтримуються браузери Chrome 90+, Safari 14+, Firefox 88+. Підтримувані мобільні платформи: iOS 13+, Android 8+. Сканування QR-коду здійснюється стандартною камерою без встановлення додаткових застосунків.

2.2.5 Вимоги до логування

Усі HTTP-запити та WebSocket-події логуються у структурованому JSON-форматі з обов'язковими полями: timestamp, request_id для наскрізного трасування, тип події, статус-код. Фінансові операції записуються в окремий незмінний audit log з мінімальним терміном зберігання 3 роки. Особисті дані (паролі, токени, платіжні дані) не потрапляють до логів.

2.2.6 Вимоги до моніторингу

Система надає health-check ендпоінт із перевіркою стану всіх критичних залежностей (база даних, платіжний шлюз, OAuth-сервіс). Метрики збираються в реальному часі: кількість активних з'єднань, час відповіді, частота помилок. При перевищенні порогів автоматично формуються алерти.

2.3 Use Case діаграми системи за ролями

2.3.1 Use Case діаграма ролі «Гість»

На рисунку 2.1 зображено use case діаграму для ролі «Гість», що охоплює два режими користування: анонімний (доступ через сканування QR-коду без реєстрації) та авторизований (з активним обліковим записом, створеним через email-реєстрацію або Google OAuth). Анонімний гість має доступ до базового циклу замовлення — сканування QR, перегляд меню з кастомізацією страв, формування

кошика з групами подачі, відстеження статусу приготування, виклик офіціанта та оплата. Авторизований гість додатково отримує доступ до історії замовлень, залишення відгуків та керування власним профілем.

Рисунок 2.1 — Use Case діаграма ролі «Гість» (анонімний та авторизований)

2.3.2 Use Case діаграма ролі «Офіціант»

На рисунку 2.2 зображено use case діаграму для ролі «Офіціант». Основним робочим простором офіціанта є інтерактивна карта залу, де відображається поточний стан кожного столика. Через спеціалізовану панель офіціант підтверджує виклики гостей (загальні та для готівкової оплати), додає позиції до активних замовлень за зверненням гостя, скасовує замовлення з обов'язковим зазначенням причини, закриває столики вручну та вмикає режим аварійного відновлення сесії гостя.

Рисунок 2.2 — Use Case діаграма ролі «Офіціант»

2.3.3 Use Case діаграма ролі «Кухар»

На рисунку 2.3 зображено use case діаграму для ролі «Кухар». Кухар взаємодіє з системою виключно через інтерфейс кухонного дисплея (KDS), де отримує нові замовлення в режимі реального часу через WebSocket. Сфера відповідальності кухаря охоплює керування статусами груп подачі за моделлю waiting → cooking → ready → served, перемикання між режимами відображення Order View та Table View, а також управління стоп-листом — додавання та зняття позицій із меню, інгредієнтів, додатків.

Рисунок 2.3 — Use Case діаграма ролі «Кухар»

2.3.4 Use Case діаграма ролі «Адміністратор»

На рисунку 2.4 зображено use case діаграму для ролі «Адміністратор». Адміністратор має повний доступ до всіх модулів системи: управління меню (CRUD для категорій, страв, інгредієнтів, додатків та груп компонентів), управління персоналом (створення акаунтів, призначення ролей, скидання паролів, деактивація), управління столиками та QR-кодами, перегляд аналітичного дашборду, генерація PDF-меню, модерація відгуків та налаштування параметрів ресторану. Окрім адміністративних функцій, адміністратор також має доступ до операційних інтерфейсів офіціанта та кухаря.

Рисунок 2.4 — Use Case діаграма ролі «Адміністратор»

2.4 Вимоги до тарифної моделі

Система реалізує модель Freemium з технічним розподілом функціоналу на рівні бази даних та middleware. Кожен ресторан характеризується значенням поля `plan` $\in \{\text{free}, \text{premium}\}$, яке визначає набір доступних функцій. Детальний опис розподілу функціоналу між тарифами наведено у п. 1.8 цієї роботи.

З точки зору формування вимог тарифний розподіл породжує такі додаткові функціональні вимоги:

- REQ-PLAN-01: Серверна частина перевіряє значення `restaurant.plan` перед обробкою запитів до тарифно-обмежених ендпоінтів (`/payments/initiate`, `/admin/analytics/*`, `/admin/menu/pdf`, `/admin/reviews`, `/admin/staff` для додавання понад 3 акаунтів). При недостатньому рівні підписки повертається HTTP 403 з кодом помилки `PLAN_REQUIRED`.

- REQ-PLAN-02: Серверна частина перевіряє кількісні обмеження для тарифу `free` при операціях створення сутностей: максимум 5 категорій меню, 50 страв, 3

зображення на страву, 3 акаунти персоналу сумарно. При перевищенні ліміту повертається HTTP 403 з кодом PLAN_LIMIT_EXCEEDED.

– REQ-PLAN-03: Клієнтська частина отримує значення plan у складі профілю ресторану та умовно рендерить елементи інтерфейсу: для тарифу free приховуються пункти меню «Аналітика», «Управління відгуками», «Генератор PDF», «Інтеграція платежів», а на формах редагування меню відображається залишок ліміту.

– REQ-PLAN-04: При спробі гостя залишити відгук для замовлення в ресторані з тарифом free система не відображає форму відгуку та не активує відповідну кнопку.

– REQ-PLAN-05: Інтеграція з Google Translate (для автоматичного перекладу полів вводу) активується лише для тарифу premium.

3 АРХІТЕКТУРА ТА ПРОЄКТУВАННЯ

3.1 Загальна архітектура системи

Програмна система реалізована за класичною трирівневою архітектурою (three-tier architecture) клієнт-сервер з розділенням на рівень представлення, рівень бізнес-логіки та рівень зберігання даних. Така архітектура є усталеним рішенням для веб-застосунків і забезпечує чітке розділення відповідальностей між компонентами, що, у свою чергу, підвищує супроводжуваність, тестованість та масштабованість системи.

На рисунку 3.1 зображено загальну архітектурну схему системи з усіма основними компонентами та зовнішніми залежностями.

Рисунок 3.1 — Загальна архітектурна схема системи QR Restaurant

Рівень представлення реалізований у вигляді односторінкового застосунку (Single Page Application, SPA) на основі бібліотеки ReactJS. Один і той самий кодовий базис обслуговує три типи користувацьких інтерфейсів — мобільний інтерфейс гостя (із підтримкою Progressive Web App), панель персоналу (офіціант, кухар) та адміністративну панель — з умовним рендерингом залежно від ролі автентифікованого користувача. Розгортання клієнтської частини здійснюється у вигляді статичних артефактів (HTML, JavaScript, CSS), що віддаються через CDN.

Рівень бізнес-логіки реалізований як серверний застосунок на платформі Node.js 18 LTS із використанням фреймворку Express.js. Сервер виконує дві основні функції: обробляє REST API-запити від клієнтських застосунків та підтримує WebSocket-з'єднання для двосторонньої передачі подій у режимі реального часу. Усі вхідні запити проходять конвеєр проміжного програмного забезпечення (middleware): автентифікація, авторизація (RBAC), перевірка тарифного плану, валідація вхідних даних, обробка помилок.

Рівень зберігання даних реалізований на основі документно-орієнтованої бази даних MongoDB 6.0+. Обґрунтування вибору NoSQL-рішення детально розглянуто у п. 3.4. Файлові ресурси (зображення страв) зберігаються в об'єктному сховищі AWS S3 з окремим bucket на кожен ресторан.

Зовнішніми сервісами системи є: LiqPay (платіжний шлюз для онлайн-оплати), Google OAuth 2.0 (автентифікація через Google-акаунт), Resend (доставка транзакційних email-повідомлень), Google Translate API (опціональний переклад двомовних полів для тарифу Premium).

Архітектурний стиль системи можна охарактеризувати як multi-tenant SaaS (програмне забезпечення як послуга з підтримкою множинної оренди): одна інстанція системи обслуговує багатьох клієнтів-ресторанів з повною ізоляцією даних на рівні префіксу restaurantId у всіх API-маршрутах та індексованих полів у документах MongoDB. Така архітектура є економічно ефективною з точки зору споживання ресурсів та спрощує процес розгортання оновлень одночасно для всіх клієнтів.

3.2 Архітектура клієнтської частини

3.2.1 Технологічний стек та інструменти збірки

Клієнтська частина побудована на базі бібліотеки ReactJS версії 18 з використанням функціональних компонентів та хуків (hooks) [9]. Збірка та локальна розробка виконується інструментом Vite — швидким збиральником на основі ES-модулів, що забезпечує миттєве оновлення модулів під час розробки (Hot Module Replacement) та оптимізовану продакшен-збірку з tree-shaking та code-splitting.

Маршрутизація реалізована через React Router v6 з підтримкою захищених маршрутів (Protected Routes), які перевіряють роль автентифікованого користувача перед монтуванням сторінки. Інтернаціоналізація забезпечується бібліотекою i18next з ресурсами української та англійської мов, організованими за принципом «один JSON-файл на сторінку».

3.2.2 Структурна організація компонентів

Структуру клієнтського застосунку зображено на рисунку 3.2.

Рисунок 3.2 — Структура клієнтської частини: ієрархія сторінок та контекстів

Каталог `src/pages` містить кореневі сторінки, згруповані за призначенням: `pages/client` (гостьовий інтерфейс), `pages/staff` (панель персоналу), `pages/onboarding` (реєстрація нового ресторану), `pages/Login`. Кожна сторінка є самостійним React-компонентом, обгорнутим у `ProtectedRoute` з зазначенням переліку дозволених ролей.

Каталог `src/components` містить багаторазові компоненти інтерфейсу, серед яких базові примітиви (`InputField`, `Dropdown`, `PrimaryButton`), складені UI-блоки (`StaffShell` — каркас сторінок персоналу з бічним меню), а також спеціалізовані компоненти для real-time повідомлень (`NotificationToast`, `HttpErrorToast`, `WsStatusChip`).

3.2.3 Управління станом через контексти React

Стан застосунку інкапсульований у глобальних контекстах React, що дозволяє уникнути «провальювання» властивостей крізь дерево компонентів (`prop drilling`) та забезпечує єдину точку доступу до спільних даних [9].

`AuthContext` інкапсулює стан автентифікації: профіль користувача, JWT-токени, методи `login`, `logout`, `updateProfile`, `requestEmailChange`, `deleteAccount`. Контекст автоматично оновлює токени за допомогою `refresh`-механізму та опрацьовує сценарій закінчення терміну дії `access`-токена.

`AppContext` інкапсулює стан гостьової сесії: токен сесії, ідентифікатор столика та ресторану, активне замовлення, єдине `WebSocket`-з'єднання, чергу непрочитаних сповіщень. Цей контекст є центральною точкою для real-time

оновлень: при отриманні WebSocket-події він оновлює внутрішній стан, що автоматично призводить до перерендерингу всіх компонентів-споживачів.

ThemeContext керує темою інтерфейсу (світла / темна) з persistence у localStorage та підтримкою системних налаштувань (prefers-color-scheme). ClientToastContext надає глобальний механізм відображення toast-повідомлень із чергою та автоматичним прихованням. Повний перелік контекстів та їх реалізацію розглянуто у підрозділі 4.2.

3.2.4 Архітектура WebSocket-клієнта

Підключення до WebSocket-сервера інкапсульоване в межах AppContext як singleton на час життя сесії. Один WebSocket використовується всіма компонентами, що передплачують на події (через хук useWebSocket), що дозволяє уникнути дублювання з'єднань та зменшує навантаження на сервер. Реалізовано автоматичне відновлення з'єднання за схемою експоненційного відступу (exponential backoff) та механізм event replay: при відновленні з'єднання клієнт надсилає last_event_id і отримує всі пропущені події.

3.2.5 Підтримка PWA та офлайн-режиму

Клієнтський застосунок зареєстрований як Progressive Web App через файл manifest.json та Service Worker. Service Worker реалізує дві стратегії кешування: cache-first для статичних ресурсів (JS, CSS, шрифти, іконки) та network-first з fallback для даних меню. Кошик гостя зберігається в localStorage та відновлюється при повторному відкритті браузера. Замовлення, надіслане в офлайн-режимі, ставиться в чергу та автоматично доставляється на сервер після відновлення мережі.

3.3 Архітектура серверної частини

3.3.1 Технологічний стек та модульна організація

Серверна частина побудована на платформі Node.js 18 LTS з фреймворком Express.js для обробки HTTP-запитів та бібліотекою ws для підтримки WebSocket-з'єднань. Для роботи з MongoDB використовується ODM-бібліотека Mongoose, що надає рівень моделей даних із валідацією схем та автоматичним керуванням зв'язками між колекціями.

Серверний код організований за принципом шарової архітектури з чітким розділенням відповідальностей. Структуру каталогів та потік обробки запиту зображено на рисунку 3.3.

Рисунок 3.3 — Шарова архітектура серверної частини та потік обробки HTTP-запиту

Шарова архітектура серверу включає такі рівні:

- Routes (маршрути) — каталог `src/routes`. Тонкий шар, що відповідає за оголошення URL-маршрутів та делегування обробки до контролерів. Згрупований за бізнес-доменами: `auth.js`, `orders.js`, `menu.js`, `payments.js`, `waiterCalls.js`, `analytics.js` тощо.

- Middleware (проміжне програмне забезпечення) — каталог `src/middleware`. Містить функції, що виконуються до обробника маршруту: `requireAuth` (перевірка JWT-токена), `requireRole(['admin'])` (перевірка ролі), `requirePlan('premium')` (перевірка тарифного плану), `validateBody(schema)` (валідація вхідних даних за схемою `Zod`), `errorHandler` (централізована обробка помилок).

- Services (бізнес-логіка) — каталог `src/services`. Інкапсують бізнес-правила, незалежні від HTTP-протоколу. Ключові сервіси: `orderService` (створення, оновлення, скасування замовлень, перерахунок статусу столика), `notificationService`

(формування та доставка сповіщень персоналу), `paymentWebhookService` (обробка callback від LiqPay), `wsService` (broadcast WebSocket-подій до відповідних кімнат).

- `Models` (моделі даних) — каталог `src/models`. Mongoose-схеми колекцій MongoDB з валідацією, індексами та віртуальними полями. Деталі моделей розглянуто у п. 3.4.

- `Utils` (допоміжні функції) — генерація short-code, хешування паролів, формування JWT-токенів, перевірка підпису LiqPay.

3.3.2 REST API та принципи проєктування

REST API спроектовано відповідно до принципів RESTful: ресурсо-орієнтовані URL, використання HTTP-методів за призначенням (GET для читання, POST для створення, PUT/PATCH для оновлення, DELETE для видалення), стандартні HTTP-коди стану. Усі ресурси ресторану використовують префікс `/:restaurantId/` для повної ізоляції даних: запит до замовлення одного ресторану не може випадково повернути дані іншого. Повну специфікацію API наведено у документі SRS і налічує понад 60 ендпоінтів.

Формат відповідей уніфікований: успішна відповідь містить поля `data` та `meta` (з `request_id` для трасування), помилка — поля `error.code` та `error.message`. Пагіновані відповіді додатково містять об'єкт `pagination` з полями `page`, `limit`, `total`, `pages`.

3.3.3 Middleware-конвеєр обробки запитів

Кожен вхідний HTTP-запит проходить наступний конвеєр (приклад для захищеного ендпоінта аналітики):

- `cors` — налаштування заголовків CORS;
- `bodyParser` — парсинг JSON-тіла запиту;
- `requestId` — генерація унікального ідентифікатора запиту для трасування;
- `requireAuth` — перевірка JWT та витяг профілю користувача;
- `requireRole(['admin'])` — перевірка ролі;

- `requirePlan('premium')` — перевірка тарифу ресторану;
- `validateQuery(analyticsQuerySchema)` — валідація query-параметрів;
- контролер маршруту — виконання бізнес-логіки через виклик сервісу;
- `errorHandler` — централізована обробка винятків та формування відповіді.

3.3.4 Реалізація розподілу тарифів через middleware

Тарифне розмежування функціоналу інкапсульоване в окреме middleware `requirePlan(plan)`, що перевіряє значення поля `plan` у документі ресторану перед обробкою запиту. Аналогічно реалізовано middleware `checkResourceLimit(resource)`, що перевіряє кількісні обмеження безкоштовного тарифу при операціях створення нових сутностей (категорій, страв, акаунтів персоналу). Такий підхід забезпечує єдину точку контролю тарифних обмежень і дозволяє легко змінювати правила без модифікації кожного окремого контролера. Реалізацію цього middleware детально розглянуто у підрозділі 4.5.

3.4 Проєктування бази даних

3.4.1 Обґрунтування вибору MongoDB

Вибір документно-орієнтованої бази даних MongoDB обумовлений природою предметної моделі системи. Меню ресторану має ієрархічну структуру з вкладеними сутностями (страва містить інгредієнти, добавки, групи компонентів), що природно представляється у форматі JSON-документа без необхідності множинних JOIN-операцій [10]. Гнучка схема MongoDB дозволяє оперативно вносити зміни до структури даних без виконання міграцій, що є критичним для проєкту в активній фазі розвитку. Крім того, MongoDB надає вбудовану підтримку TTL-індексів (автоматичне видалення документів за певний час), що використовується для очищення прострочених сесій, токенів скидання пароля та сповіщень [10].

3.4.2 Огляд ключових колекцій

Domain-модель системи з усіма основними сутностями та зв'язками між ними наведено на рисунку 3.4. Нижче наведено короткий опис ключових колекцій із зазначенням стратегії індексування.

Рисунок 3.4 — Діаграма класів (Domain Model): основні сутності системи QR Restaurant

Перелік ключових колекцій бази даних:

- restaurants — кореневі документи ресторанів. Містить поля name, address, plan, enabledLanguages, settings. Поле plan є ключовим для системи тарифного розподілу функціоналу.
- tables — столики ресторану. Унікальний індекс на shortCode забезпечує миттєвий пошук при скануванні QR. Складений індекс на (restaurantId, status) оптимізує запити карти залу для офіціанта.
- sessions — гостьові сесії, прив'язані до конкретного столика. TTL-індекс на полі expiresAt (2 години) автоматично видаляє прострочені сесії.
- orders — замовлення гостей. Складений індекс на (tableId, status) критичний для перевірки обмеження одного активного замовлення на столик.
- orderItems та servingGroups — позиції замовлення та групи подачі.
- menuItems, categories, ingredients, addOns, componentGroups — сутності меню з підтримкою soft-delete через поле isDeleted.
- waiterCalls — виклики офіціанта з полями type (call або cash_payment) та status (active або resolved).
- payments — записи фінансових транзакцій із посиланням на замовлення та методом оплати.
- reviews — відгуки гостей (RestaurantReview та DishReview як єдиний поліморфний документ із полем type).

- notifications — сповіщення персоналу з TTL-індексом на 7 діб.
- auditLog — незмінний журнал критичних операцій (скасування замовлень, готівкові оплати, walk-out) з мінімальним терміном зберігання 3 роки.
- users — єдина колекція для всіх ролей із полем role та опціональним googleId (sparse-індекс).

3.4.3 Стратегія цілісності даних

Оскільки MongoDB не підтримує foreign key constraints, цілісність зв'язків між сутностями забезпечується на рівні застосунку через Mongoose-проміжне програмне забезпечення (pre/post hooks) та явні перевірки в сервісному шарі [10]. Атомарність складних операцій (наприклад, створення замовлення з оновленням статусу столика) реалізована через MongoDB-транзакції на репліка-сеті.

3.4.4 Діаграма стану замовлення

Сутність Order має складний життєвий цикл із чітко визначеним набором станів та однонаправлених переходів між ними. На рисунку 3.5 зображено діаграму стану замовлення, яка відображає всі можливі статуси та умови їх зміни.

Рисунок 3.5 — Діаграма стану замовлення (Order State Diagram)

Початковим станом замовлення є open — замовлення підтверджено гостем та передано на кухню; цей статус не залежить від стадій приготування окремих страв і зберігається до явної фінансово-операційної дії. Зі статусу open можливі три переходи: до open_raided (при підтвердженні готівкової або онлайн-оплати до закриття столика), до completed_cash або completed_pay (при закритті замовлення офіціантом разом із підтвердженням типу оплати) та до cancelled (скасування замовлення офіціантом або адміністратором з обов'язковим зазначенням причини). Стан open_raided є проміжним: він фіксує факт здійсненої оплати, але дозволяє продовжити обслуговування столика; із нього замовлення переходить до

фінального стану `completed_*` при закритті столика. Стани `cancelled`, `completed_cash` та `completed_pay` є термінальними — після переходу до них будь-які зміни замовлення заблоковано (HTTP 409).

3.5 Архітектура взаємодії в реальному часі

3.5.1 Модель кімнат та підписок

Передача подій у реальному часі реалізована через WebSocket-протокол з організацією учасників за моделлю кімнат (rooms). Кімната — це логічна група з'єднань, що отримують одні й ті самі події. Система підтримує чотири типи кімнат:

- `restaurant:{restaurantId}` — широкомовні події для всього персоналу ресторану (зміни меню);
- `table:{tableId}` — події конкретного столика (зміни статусу, виклики офіціанта);
- `session:{sessionToken}` — персональні події гостя (статус власного замовлення, підтвердження оплати);
- `kitchen:{restaurantId}` та `waiter:{restaurantId}` — рольові кімнати персоналу.

При встановленні WebSocket-з'єднання сервер автоматично приєднує учасника до відповідних кімнат на основі типу автентифікації: гість потрапляє до `session:{token}` та `table:{tableId}`, кухар — до `kitchen:{restaurantId}`, офіціант — до `waiter:{restaurantId}`. Архітектуру кімнат зображено на рисунку 3.6.

Рисунок 3.6 — Архітектура WebSocket-кімнат та потоки подій

3.5.2 Каталог подій та формат повідомлень

Усі повідомлення мають уніфікований формат із обов'язковими полями `event`, `payload`, `timestamp` та `event_id`. Поле `event_id` є унікальним ідентифікатором події, що використовується для механізму відтворення. Повний каталог подій налічує 14

типів серверних подій (наприклад, ORDER_NEW, DISH_STATUS_UPDATED, WAITER_CALL, PAYMENT_COMPLETED).

3.5.3 Механізм event replay та відновлення з'єднання

При розриві WebSocket-з'єднання клієнт автоматично виконує повторні спроби підключення з експоненційними паузами (3 секунди, до 5 спроб). При успішному відновленні з'єднання клієнт надсилає повідомлення REPLAY_REQUEST із значенням last_event_id — ідентифікатором останньої успішно обробленої події. Сервер зберігає чергу подій тривалістю 10 хвилин на кожну кімнату і відтворює всі події, що надійшли після зазначеного event_id. Якщо розрив тривав довше 10 хвилин, клієнт виконує повне перезавантаження даних через REST API.

Поточний стан з'єднання відображається у компоненті WsStatusChip у вигляді кольорового індикатора (Online / Reconnecting / Offline), що підвищує прозорість системи для користувача.

3.6 Інтеграція з зовнішніми сервісами

3.6.1 Інтеграція з платіжним шлюзом LiqPay

LiqPay використовується для онлайн-оплати банківськими картками. Інтеграція реалізована за стандартною схемою з ініціацією оплати на клієнтській стороні та підтвердженням транзакції через webhook на серверній стороні. Послідовність взаємодії: гість натискає кнопку «Оплатити онлайн» — клієнт надсилає POST `/:restaurantId/payments/initiate { orderId }`; сервер створює запис Payment зі статусом pending, формує payload із даними замовлення та підписує його HMAC-SHA1 з використанням секретного ключа LiqPay; клієнт відкриває форму LiqPay із підписаним payload; гість завершує оплату на стороні LiqPay; LiqPay надсилає асинхронний webhook на POST `/api/payments/webhook/liqpay` з результатом транзакції; сервер перевіряє підпис webhook, оновлює статус

Payment.status та переводить замовлення у статус open_paid з фіксацією методу оплати eraу; сервер генерує WebSocket-подію PAYMENT_COMPLETED для гостя та персоналу.

Реалізовано fallback-механізм для випадку ненадходження webhook: серверне завдання за розкладом виконує повторні запити статусу до LiqPay API через 1, 5 та 15 хвилин після ініціації. Інтеграція доступна виключно для ресторанів із тарифом Premium.

3.6.2 Інтеграція з Google OAuth 2.0

Google OAuth 2.0 надає альтернативний метод автентифікації для гостей та персоналу. Інтеграція реалізована через бібліотеку Passport.js із стратегією passport-google-oauth20. Користувач натискає «Увійти через Google» — клієнт перенаправляє на GET /auth/google; сервер ініціює OAuth-flow та перенаправляє на сторінку Google; після підтвердження користувачем Google перенаправляє на GET /auth/google/callback?code=...; сервер обмінює code на access-токен Google, отримує профіль користувача (email, ім'я, googleId); сервер перевіряє наявність акаунту з відповідним googleId або email: якщо акаунт існує — виконує вхід, якщо ні — створює новий акаунт; сервер видає JWT access- та refresh-токени та повертає клієнту. Підтримується flow прив'язування Google до існуючого email-акаунта та відв'язування за умови наявності альтернативного методу входу.

3.6.3 Інтеграція з AWS S3 для зберігання зображень

Зображення страв зберігаються в об'єктному сховищі AWS S3 з окремим bucket на кожен ресторан. Завантаження виконується через серверну частину: клієнт надсилає multipart-форму на POST /:restaurantId/admin/menu/items/{itemId}/image, сервер валідує MIME-тип (JPG, PNG, WebP) та розмір (≤ 5 МБ), після чого виконує upload до S3 та зберігає публічний URL у документі страви. Такий підхід забезпечує контроль над вхідним контентом та можливість попередньої обробки зображень (resize, оптимізація).

3.6.4 Інтеграція з Resend для транзакційних email

Сервіс Resend використовується для доставки трьох типів транзакційних email-повідомлень: підтвердження email при onboarding нового ресторану, скидання пароля, підтвердження зміни email-адреси. Шаблони листів реалізовані як React-компоненти з використанням бібліотеки react-email, що дозволяє підтримувати їх у тому самому форматі, що й основний клієнтський інтерфейс.

3.6.5 Інтеграція з Google Translate API

Для ресторанів із тарифом Premium активується інтеграція з Google Translate API для автоматичного перекладу двомовних полів вводу (назв та описів категорій, страв, інгредієнтів). При редагуванні поля адміністратор може натиснути кнопку «Auto-translate», після чого клієнт надсилає запит на проксі-ендпоінт POST `/:restaurantId/admin/translate` (для приховування API-ключа від клієнта), а сервер виконує запит до Google Translate API і повертає перекладений текст.

3.7 Безпека та автентифікація

3.7.1 Модель автентифікації

Система підтримує два типи автентифікації. JWT-автентифікація для авторизованих користувачів (гостей з акаунтом та персоналу): access-токен має термін дії не більше 1 години, refresh-токен — 30 діб. Токени передаються через заголовок `Authorization: Bearer {token}`. При закінченні access-токена клієнт автоматично виконує refresh через POST `/auth/refresh`.

Session-токен для анонімних гостей: криптографічно стійкий випадковий токен (32 байти, base64-кодування), згенерований при скануванні QR-коду та збережений у HttpOnly Secure cookie з атрибутами SameSite=Strict та Max-Age=86400 (24 години).

3.7.2 Авторизація через RBAC

Авторизація реалізована за моделлю Role-Based Access Control. Кожен користувач має одну з ролей: guest, cook, waiter, waiter_cook, admin. Перевірка ролі виконується `middleware requireRole([...])` для кожного захищеного ендпоінта. Додатково реалізована перевірка приналежності до ресторану: користувач не може виконати операцію над сутностями ресторану, до якого він не належить (HTTP 403).

3.7.3 Захист від типових атак

Реалізовано стандартні механізми захисту:

- XSS: усі вхідні дані екрануються; CSP-заголовки обмежують джерела завантаження ресурсів.
- CSRF: для всіх POST/PUT/PATCH/DELETE запитів використовується `SameSite=Strict` cookie + перевірка Origin заголовка.
- NoSQL-ін'єкції: вхідні дані валідуються через схеми Zod перед передачею в Mongoose-запити; оператори типу `$gt`, `$ne` в полях запиту блокуються.
- Rate limiting: ендпоінт POST /auth/login обмежений до 5 спроб за 15 хвилин на IP-адресу для запобігання brute-force атакам.
- Захист webhook: усі webhook від LiqPay перевіряються через HMAC-SHA1 підпис з секретним ключем.

3.7.4 Зберігання паролів та секретів

Паролі користувачів зберігаються виключно у вигляді bcrypt-хешів з параметром `salt-rounds ≥ 12`. Токени скидання пароля та підтвердження зміни email зберігаються у вигляді SHA-256 хешу, що унеможливило використання токена навіть у разі компрометації бази даних. Секретні ключі (LiqPay secret, JWT secret, AWS credentials, Google Auth secrets) зберігаються у захищеному сховищі секретів і ніколи не потрапляють до коду чи системи контролю версій.

3.8 Проєктування інтерфейсу користувача

Проєктування інтерфейсу системи підпорядковане принципу мінімізації кількості дій, необхідних гостю для оформлення замовлення, що відповідає нефункціональній вимозі зручності (виконання першого замовлення без інструкцій за не більше ніж три дії). Інтерфейс побудовано за підходом *mobile-first*: основним каналом взаємодії є смартфон гостя, тому базовою точкою проєктування обрано екран шириною від 320 px, з подальшою адаптацією під більші екрани персоналу та адміністратора.

Система обслуговує три принципово різні типи інтерфейсів у межах єдиної кодової бази. Гостьовий інтерфейс оптимізований під вертикальну орієнтацію смартфона та сенсорне керування: великі області натискання, нижня навігаційна панель, мінімум текстового вводу. Інтерфейс персоналу (панель офіціанта, кухонний дисплей) розрахований на планшети та горизонтальну орієнтацію — карта залу та картки KDS використовують сітку (*grid*) із крупними елементами, придатними для швидкої взаємодії в умовах кухні. Адміністративна панель орієнтована на настільні екрани та містить таблиці, форми та аналітичні графіки.

Для забезпечення візуальної консистентності всі екрани побудовані з єдиного набору багаторазових компонентів (*InputField*, *Dropdown*, *PrimaryButton* тощо) та системи дизайн-токенів, реалізованої через CSS-змінні. Підтримуються світла та темна теми, перемикання між якими відбувається через *ThemeContext* із збереженням вибору в *localStorage* та врахуванням системного налаштування *prefers-color-scheme*. Поточний стан *real-time* з'єднання та системні події відображаються через уніфіковані компоненти зворотного зв'язку (*NotificationToast*, *HttpErrorToast*, *WsStatusChip*), що забезпечує прозорість стану системи для користувача.

Окрему увагу приділено індикації станів очікування та помилок: кожна асинхронна операція (завантаження меню, підтвердження замовлення, оплата) супроводжується станами завантаження (*skeleton/spinner*) та зрозумілими

повідомленнями про помилки українською мовою, що знижує невизначеність для користувача та підвищує довіру до системи.

3.9 Сценарії взаємодії компонентів системи

Для ілюстрації динамічної взаємодії між компонентами системи, описаними у попередніх підрозділах, наведено три sequence-діаграми ключових сценаріїв: повний цикл обслуговування гостя без помилок, обробка помилок платіжного шлюзу та механізм відновлення WebSocket-з'єднання після розриву. Ці сценарії охоплюють як основний потік (happy path), так і критичні випадки відмов, що дозволяє оцінити взаємодію всіх архітектурних шарів у комплексі.

3.9.1 Повний цикл обслуговування гостя

На рисунку 3.7 зображено sequence-діаграму повного циклу взаємодії гостя із системою у нормальному сценарії без помилок: сканування QR-коду → виконання Smart Session Recovery або генерація нової сесії → перегляд меню з гостьовою кастомізацією страв → формування кошика з групами подачі → підтвердження замовлення з передачею на KDS через WebSocket-подію ORDER_NEW → отримання real-time оновлень статусів страв через DISH_STATUS_UPDATED → ініціація онлайн-оплати через LiqPay → отримання webhook-підтвердження та переведення замовлення у статус open_paid. Діаграма демонструє послідовну взаємодію клієнтського SPA, REST API, WebSocket-сервера, бази даних MongoDB та зовнішнього платіжного шлюзу.

Рисунок 3.7 — Sequence-діаграма повного циклу обслуговування гостя

3.9.2 Обробка помилок платіжного шлюзу

На рисунку 3.8 зображено sequence-діаграму обробки помилкових сценаріїв при взаємодії з платіжним шлюзом LiqPay. Розглянуто чотири типи відмов: timeout

відповіді шлюзу при ініціації платежу, явна відмова шлюзу під час перевірки картки, відхилений платіж (недостатньо коштів, заблокована картка) та ненадходження webhook-підтвердження протягом визначеного часу. Для останнього випадку відображено fallback-механізм: серверне завдання за розкладом виконує повторні запити статусу транзакції через 1, 5 та 15 хвилин після ініціації, що забезпечує консистентність між станом замовлення в системі та фактичним результатом транзакції на стороні шлюзу.

Рисунок 3.8 — Sequence-діаграма обробки помилок платіжного шлюзу та fallback-механізму

3.9.3 Відновлення WebSocket-з'єднання та event replay

На рисунку 3.9 зображено sequence-діаграму механізму відновлення WebSocket-з'єднання після розриву, що є критичним для забезпечення цілісності real-time-комунікації між гостем, кухнею та офіціантом. Сценарій охоплює: виявлення клієнтом розриву з'єднання, автоматичні повторні спроби підключення з експоненційними паузами, надсилання повідомлення `REPLAY_REQUEST` із значенням останньої успішно обробленої події (`last_event_id`), пошук пропущених подій сервером у in-memory черзі з TTL 10 хвилин та послідовне відтворення пропущених подій клієнту. У випадку, коли розрив тривав довше за термін зберігання черги, клієнт виконує повне перезавантаження стану через REST API.

Рисунок 3.9 — Sequence-діаграма відновлення WebSocket-з'єднання та механізму event replay

4 ОПИС ПРИЙНЯТИХ ПРОГРАМНИХ РІШЕНЬ

У цьому розділі розглянуто ключові програмні рішення, прийняті при реалізації системи, з наведенням фрагментів реального коду. Основну увагу приділено тим механізмам, що визначають нефункціональні характеристики системи: єдиному WebSocket-з'єднанню, підтримці багатомовності, технічному розподілу тарифів, офлайн-режиму та системі сповіщень персоналу.

4.1 Структура коду проєкту

Проєкт організовано як два незалежні застосунки: клієнтська частина (каталог QR Rest) та серверна частина (каталог QR Rest API). Клієнтська частина побудована на Vite-збірці та містить такі ключові каталоги: `src/pages` (сторінки, згруповані за ролями), `src/components` (багаторазові компоненти інтерфейсу), `src/context` (дев'ять глобальних контекстів React), `src/hooks` (користувацькі хуки, зокрема `useWebSocket` та `usePlan`), `src/i18n` (інфраструктура багатомовності), `src/utlis` (допоміжні модулі, зокрема черга офлайн-замовлень) та `src/api` (обгортки HTTP-запитів на основі `Axios`).

Серверна частина організована за шаровою архітектурою (детально розглянуто у підрозділі 3.3) із каталогами `src/routes`, `src/middleware`, `src/services`, `src/models` та `src/utlis`, а також каталогом `scripts`, що містить службові скрипти, зокрема `db-seed.js` для наповнення бази даних тестовими даними.

4.2 Управління станом через контексти React

Глобальний стан клієнтської частини інкапсульований у дев'яти контекстах React, кожен з яких відповідає за окрему предметну область, що відповідає принципу єдиної відповідальності та спрощує супроводжуваність коду [9]. Контексти об'єднані в ієрархію провайдерів, у якій зовнішні провайдери надають дані внутрішнім.

Перелік реалізованих контекстів:

- AuthContext — стан автентифікації (профіль користувача, JWT-токени, методи login, logout, deleteAccount).
- AppContext — центральний контекст (близько 1200 рядків коду): кошик, життєвий цикл замовлення, відновлення активного замовлення при вході, метадані ресторану, єдине WebSocket-з'єднання та сесія гостя.
- StaffDataContext — кешовані ресурси панелі персоналу (столики, категорії, страви, персонал) із ледачим завантаженням та узгодженням через WebSocket.
- PlanContext — похідний контекст тарифного плану ресторану (plan, isPremium).
- MenuContext — клієнтський кеш меню з обмеженням актуальності даних (5 хвилин для списку, 3 хвилини для деталей страви).
- ThemeContext — керування світлою / темною темою.
- ClientToastContext — глобальні toast-повідомлення для гостьового інтерфейсу.
- StaffNotificationsContext — сповіщення персоналу в реальному часі (розглянуто у підрозділі 4.7).
- StaffLayoutContext — стан бічної панелі сповіщень у каркасі сторінок персоналу.

Універсальний хук usePlan інкапсулює логіку визначення тарифу: він спершу звертається до PlanContext (для персоналу), потім до поля restaurantPlan у AppContext (для гостьового потоку), а за відсутності даних повертає значення free за замовчуванням. Такий підхід дозволяє використовувати єдиний інтерфейс перевірки тарифу як на сторінках персоналу, так і в гостьовому інтерфейсі.

4.3 Архітектура єдиного WebSocket-з'єднання

Для уникнення дублювання з'єднань реалізовано єдине глобальне WebSocket-з'єднання, що належить AppContext і використовується всіма компонентами через тонкий хук-обгортку useWebSocket. До рефакторингу кожна сторінка відкривала

власне з'єднання, що для користувача-персоналу при навігації означало 4–6 паралельних з'єднань до того самого сервера. Реалізацію хука наведено у лістингу 4.1.

Лістинг 4.1 — Хук `useWebSocket`: підписка на єдине глобальне з'єднання

```
export function useWebSocket({ onMessage, enabled = true }) {
  const { wsStatus, wsLatency, wsSubscribe,
    addWsListener, removeWsListener } = useApp();
  const handlerRef = useRef(onMessage);

  // Зберігаємо актуальне посилання на обробник без повторної підписки
  useEffect(() => { handlerRef.current = onMessage; }, [onMessage]);

  useEffect(() => {
    if (!enabled) return;
    const fn = (msg) => handlerRef.current?.(msg);
    addWsListener(fn);
    return () => removeWsListener(fn);
  }, [enabled, addWsListener, removeWsListener]);

  return { status: wsStatus, latency: wsLatency, subscribe: wsSubscribe };
}
```

Кожен компонент-споживач лише реєструє власний слухач (listener) на спільному з'єднанні через `addWsListener` та автоматично відписується при демонтуванні. Використання `ref` для збереження обробника дозволяє оновлювати функцію-обробник без повторної підписки при кожному перерендерингу. Таким чином, увесь застосунок персоналу використовує одне `WebSocket`-з'єднання, що знижує навантаження на сервер та спрощує керування станом з'єднання.

4.4 Підтримка багатомовності

Багатомовність меню реалізована за схемою суфіксних полів: базове поле (наприклад, `name`) зберігає значення мовою-джерелом, а додаткові поля з мовним суфіксом (`name_en`, `name_pl`) — переклади. Хук `useLocalField` повертає функцію-локалізатор, що обирає коректне поле залежно від активної мови інтерфейсу. Реалізацію наведено у лістингу 4.2.

Лістинг 4.2 — Хук `useLocalField`: вибір локалізованого поля об'єкта

```
export function useLocalField() {
  const { i18n } = useTranslation();
```

```

return (obj, field) => {
  if (!obj) return '';
  const lang = i18n.language;

  // Мова-джерело -> базове поле (наприклад, 'name')
  if (lang === SOURCE_LANG) return obj[field] ?? '';

  // Спершу поле поточної мови, потім відкат до мови-джерела
  const localKey = fieldFor(field, lang);
  return obj[localKey] ?? obj[field] ?? '';
};
}

```

Перевага такого рішення полягає у незалежності від кількості мов: додавання нової мови не потребує зміни структури документів, а лише реєстрації нового мовного коду. За відсутності перекладу для поточної мови функція повертає значення мовою-джерелом, що гарантує відсутність порожніх полів в інтерфейсі. Окремий хук `useFallbackField` додатково повідомляє, чи показане значення є відкатом до мови-джерела, що використовується для візуальної індикації неперекладених полів в адміністративній панелі.

4.5 Реалізація тарифних обмежень

Технічний розподіл функціоналу між тарифами `free` та `premium` реалізовано на серверній стороні через окреме `middleware requirePlan`, що перевіряє значення поля `plan` у документі ресторану перед обробкою запиту до тарифно-обмеженого ендпоінта. Реалізацію наведено у лістингу 4.3.

Лістинг 4.3 — `Middleware requirePlan`: перевірка тарифного плану ресторану

```

function requirePlan(requiredPlan) {
  return function (req, res, next) {
    const plan = req.restaurant?.plan || 'free';
    if (plan === requiredPlan || (requiredPlan === 'free')) {
      return next();
    }
    return res.status(403).json({
      error: {
        code: 'PLAN_REQUIRED',
        message: `This feature requires the ${requiredPlan} plan`,
        requiredPlan, currentPlan: plan,
      },
      meta: {},
    });
  };
}

```

```
};
}
module.exports = requirePlan;
```

Middleware повертає функцію-обробник Express, яку приєднують до маршрутів преміум-функціоналу (ініціація онлайн-оплати, аналітика, генерація PDF-меню, управління відгуками). За недостатнього рівня підписки повертається відповідь HTTP 403 з машинозчитуваним кодом помилки `PLAN_REQUIRED`, який клієнтська частина використовує для відображення вікна пропозиції оновлення тарифу. Такий централізований підхід забезпечує єдину точку контролю тарифних обмежень і дозволяє додавати нові обмеження без модифікації кожного контролера окремо.

4.6 Офлайн-черга замовлень

Для підтримки офлайн-режиму (вимога SYS-PWA-04) реалізовано чергу замовлень у `localStorage`. Коли гість підтверджує замовлення без мережі, корисне навантаження запиту зберігається в черзі, а інтерфейс одразу отримує синтетичний плейсхолдер; при відновленні з'єднання `AppContext` послідовно відправляє накопичені замовлення. Черга переживає закриття та повторне відкриття браузера завдяки зберіганню в `localStorage`. Функцію додавання замовлення до черги наведено у лістингу 4.4.

Лістинг 4.4 — Функція `enqueueOrder`: постановка замовлення в офлайн-чергу

```
const KEY = 'pendingOrderQueue';

export function enqueueOrder({ restaurantId, payload }) {
  const items = readQueue();
  const entry = {
    id: uuid(),
    restaurantId,
    payload,
    queuedAt: new Date().toISOString(),
  };
  items.push(entry);
  writeQueue(items);
  return entry;
}
```

Кожен запис черги містить локально згенерований ідентифікатор (для дедуплікації та відображення в інтерфейсі), ідентифікатор ресторану, корисне навантаження запиту та час постановки в чергу. Послідовна відправка по одному запиту за раз гарантує, що сервер не отримає дублікатів замовлення для одного столика. Функції `readQueue` та `writeQueue` інкапсулюють роботу з `localStorage` із безпечним опрацюванням винятків (вимкнене сховище, перевищення квоти).

4.7 Система сповіщень персоналу за ролями

Сповіщення персоналу в реальному часі реалізовані в контексті `StaffNotificationsContext`, який підписується на єдине `WebSocket`-з'єднання та фільтрує події залежно від ролі користувача. Відповідність ролей і типів подій задана декларативно у структурі `ROLE_EVENTS`, наведеній у лістингу 4.5.

Лістинг 4.5 — Структура `ROLE_EVENTS`: фільтрація подій за роллю персоналу

```
const ROLE_EVENTS = {
  cook:      ['ORDER_NEW', 'ORDER_ITEMS_ADDED'],
  waiter:    ['ORDER_NEW', 'WAITER_CALL', 'WAITER_CALL_CASH'],
  waiter_cook: ['ORDER_NEW', 'ORDER_ITEMS_ADDED',
    'WAITER_CALL', 'WAITER_CALL_CASH'],
  admin:     ['ORDER_NEW', 'ORDER_ITEMS_ADDED', 'WAITER_CALL',
    'WAITER_CALL_CASH', 'ORDER_VOID',
    'ORDER_CANCELLED', 'PAYMENT_COMPLETED'],
};
```

При надходженні `WebSocket`-події обробник перевіряє, чи дозволений її тип для поточної ролі користувача, і лише тоді формує об'єкт сповіщення та відтворює звуковий сигнал. Такий підхід гарантує, що кухар отримує лише сповіщення про нові та доповнені замовлення, офіціант — про нові замовлення та виклики, а адміністратор — повний набір подій, включно зі скасуваннями та підтвердженнями оплати. Тексти сповіщень формуються через систему перекладів `i18next`, що забезпечує їх відображення мовою інтерфейсу персоналу.

4.8 Генератор PDF-меню

Генератор PDF-меню реалізований повністю на клієнтській стороні з використанням бібліотек `jsPDF` та `html2canvas`, що дозволяє формувати документ

без навантаження на сервер та без виклику діалогу друку браузера. Адміністратор налаштовує оформлення через панель параметрів: попередньо визначені шаблони, формат і орієнтацію сторінки, поля, мову, сімейство та розмір шрифту, кольори, перемикачі вмісту та параметри зображень, а також порядок категорій із вибором окремих страв.

Пагінація реалізована через окремий вимірювальний шар (off-screen measurement layer), який рендерить кожен елемент меню за реальною шириною контенту; розбиття на сторінки керується вимірюванням висоти елементів (offsetHeight) із застосуванням ResizeObserver та інтегрованим виправленням «осиротілих» заголовків. Підтримуються двомовні макети: послідовний (українською, потім англійською) та двоколонковий. На етапі завантаження бібліотека html2canvas захоплює кожен елемент сторінки з подвійним масштабом (2×) для якісного друку, після чого jsPDF зберігає результат як файл menu.pdf. Для ресторанів із тарифом free при відкритті сторінки одразу з'являється вікно пропозиції оновлення тарифу, що реалізує вимогу REQ-PLAN-01.

5 ТЕСТУВАННЯ

5.1 Стратегія тестування

Стратегія тестування системи базується на перевірці відповідності реалізованого функціоналу сформульованим критеріям приймання (АС), кожен з яких описаний у форматі Given/When/Then. Такий формат дозволяє безпосередньо трансформувати критерій приймання у тестовий сценарій із чітко визначеними передумовами, діями та очікуваним результатом. Тестування охоплює кілька рівнів: модульне тестування окремих сервісів і допоміжних функцій серверної частини, інтеграційне тестування REST API-ендпоінтів із реальною взаємодією з базою даних та ручне функціональне тестування клієнтського інтерфейсу за сценаріями використання.

Особливу увагу приділено перевірці критичних інваріантів системи: гарантії одного активного замовлення на столик, коректності однонаправлених переходів статусів груп подачі, дотримання тарифних обмежень на рівні middleware та цілісності фінансових операцій. Для real-time функціоналу окремо перевірялися сценарії розриву та відновлення WebSocket-з'єднання з відтворенням пропущених подій (event replay).

5.2 Інструменти тестування

Інтеграційне тестування серверної частини виконується з використанням бібліотеки `mongodb-memory-server`, яка запускає ізольований екземпляр MongoDB у пам'яті для кожного тестового прогону. Такий підхід забезпечує повну ізоляцію тестів від робочої бази даних, детермінованість результатів та високу швидкість виконання, оскільки після завершення тестів дані автоматично видаляються разом із зупинкою тимчасового екземпляра. Перед кожним тестом база наповнюється контрольованим набором даних, що дозволяє відтворювати передумови (Given) критеріїв приймання.

Для перевірки REST API застосовуються HTTP-запити до підготовлених ендпоінтів із подальшою перевіркою статус-кодів відповідей та структури тіла відповіді (поля `data`, `meta`, `error.code`). Це дозволяє безпосередньо перевіряти критерії, що оперують конкретними HTTP-кодами, зокрема повернення HTTP 409 при спробі створення другого активного замовлення або HTTP 403 при недостатньому рівні тарифу.

5.3 Тестові дані

Для відтворюваного тестування реалізовано скрипт наповнення бази даних `db-seed.js` (версія 6), що запускається командою `npm run db:seed`. Скрипт створює два ресторани з різними тарифами: безкоштовний (публічний ідентифікатор `BR5CH3OK`) та преміум (публічний ідентифікатор `PR3MIUM1`), що дозволяє перевіряти тарифно-залежну поведінку системи. Для безкоштовного ресторану створюється три акаунти персоналу (адміністратор, кухар, офіціант) та вимкнено систему відгуків; для преміум-ресторану — п'ять акаунтів (включно з комбінованою роллю `waiter_cook`) та активовано систему відгуків.

Кожен ресторан наповнюється реалістичними даними: 5 столиків, 7 категорій та 32 страви з українськими та англійськими перекладами, 50 замовлень (44 завершених, 2 скасованих, 4 активних) із групами подачі та відповідними платежами, а також 3 виклики офіціанта різних типів. Створюється 35 гостьових акаунтів. Пароль для всіх облікових записів — `'12345678'`. Активні замовлення розподілені між непересічними групами гостей, що гарантує дотримання обмеження одного активного замовлення на акаунт.

5.4 Тестування за критеріями приймання

У таблиці 5.1 наведено результати перевірки репрезентативної вибірки критеріїв приймання, що охоплюють ключові модулі системи. Для кожного критерію зазначено сценарій, очікуваний результат та статус перевірки.

Таблиця 5.1 — Результати тестування за критеріями приймання

Ідентифікатор	Сценарій	Очікуваний результат	Статус
AC-QR-01	Перше сканування активного QR-коду	HTTP 302 на /menu з новим sessionToken за ≤ 500 мс	Пройдено
AC-ORDER-03	Одночасне створення замовлення за одним столиком	Другий запит $\rightarrow$ HTTP 409; створено лише одне замовлення	Пройдено
AC-CART-01	Підтвердження кошика з 3 стравами	Замовлення з'являється на KDS за ≤ 300 мс	Пройдено
AC-KDS-05	Спроба зниження статусу групи з «ready» на «cooking»	HTTP 400; статус не змінюється	Пройдено
AC-CALL-02	Повторний виклик офіціанта протягом 60 с	Новий WaiterCall не створюється	Пройдено
AC-PAY-01	Надходження webhook підтвердження від LiqPay	Order.status $\rightarrow$ completed_epay; підтвердження за ≤ 3 с	Пройдено
AC-MGMT-02	Зміна ціни страви в активному замовленні	unitPrice в OrderItem залишається незмінним	Пройдено
AC-AUTH-01	Реєстрація нового користувача	Пароль у БД — bcrypt-хеш; акаунт активний	Пройдено
AC-PWA-03	Підтвердження замовлення в офлайн-режимі	Замовлення відправляється автоматично після відновлення мережі	Пройдено
AC-REV-02	Повторне залишення відгуку про ресторан	HTTP 409	Пройдено

5.5 Результати тестування

За результатами тестування підтверджено відповідність реалізованої системи сформульованим критеріям приймання. Усі перевірені сценарії, наведені у таблиці 5.1, завершилися успішно, що засвідчує коректність реалізації критичних інваріантів системи: обмеження одного активного замовлення на столик, однаправленості переходів статусів, дотримання тарифних обмежень, цілісності цінових знімків (price snapshot) та безпечного зберігання паролів. Окремо

підтверджено працездатність механізмів real-time взаємодії та офлайн-режиму, що є ключовими конкурентними перевагами системи. Виявлені в процесі тестування невідповідності було усунено, після чого повторне тестування підтвердило очікувану поведінку.

6 ВПРОВАДЖЕННЯ

6.1 Середовище розгортання

Система розгортається за моделлю multi-tenant SaaS: одна інстанція обслуговує багатьох клієнтів-ресторанів з ізоляцією даних на рівні префіксу restaurantId. Архітектура розгортання відповідає трирівневій структурі системи та складається з трьох логічно незалежних компонентів: статичної клієнтської частини, серверного застосунку Node.js та бази даних MongoDB. Такий поділ дозволяє масштабувати кожен компонент незалежно та забезпечує гнучкість при виборі хостинг-провайдера.

Клієнтська частина після збірки є набором статичних артефактів (HTML, JavaScript, CSS), які не потребують серверного виконання і можуть віддаватися через мережу доставки контенту (CDN) або статичний хостинг. Серверна частина розгортається як довготривалий процес Node.js, що обслуговує REST API та WebSocket-з'єднання. Файлові ресурси (зображення страв) зберігаються у зовнішньому об'єктному сховищі AWS S3, що знімає навантаження зберігання з серверного застосунку.

6.2 Конфігурація через змінні оточення

Конфігурація системи винесена у змінні оточення (environment variables), що відповідає принципу відокремлення конфігурації від коду та дозволяє розгортати ту саму збірку в різних середовищах (розробка, тестування, продакшен) без модифікації коду. Клієнтська частина використовує змінні VITE_API_URL (базова адреса REST API) та VITE_WS_URL (адреса WebSocket-сервера); за відсутності останньої адреса WebSocket виводиться автоматично з адреси API.

Серверна частина споживає змінні оточення для підключення до бази даних (рядок з'єднання MongoDB), секретних ключів JWT, облікових даних інтеграцій (LiqPay secret та public key, Google OAuth client ID та secret, AWS S3 credentials, API-

ключ Resend та Google Translate). Секретні значення ніколи не потрапляють до системи контролю версій і зберігаються у захищеному сховищі секретів середовища розгортання.

6.3 Збірка та розгортання

Збірка клієнтської частини виконується інструментом Vite командою `npm run build`, що формує оптимізований набір статичних файлів з мінімізацією, `tree-shaking` та `code-splitting`. Отримані артефакти розгортаються на статичному хостингу або CDN. Локальна розробка ведеться через команду `npm start`, що запускає dev-сервер Vite з миттєвим оновленням модулів (Hot Module Replacement).

Серверний застосунок запускається як процес Node.js, що підключається до екземпляра MongoDB (для забезпечення підтримки транзакцій — на репліка-сеті) та зовнішніх сервісів. Перед першим запуском у тестовому середовищі база даних наповнюється скриптом `db-seed.js`. Послідовність розгортання передбачає: налаштування змінних оточення, розгортання бази даних MongoDB, запуск серверного застосунку Node.js, публікацію статичної клієнтської збірки на CDN та налаштування bucket-ів AWS S3 для зберігання зображень.

6.4 Моніторинг та логування

Для забезпечення спостережуваності системи в робочому середовищі реалізовано health-check ендпоінт, що перевіряє стан критичних залежностей (база даних, платіжний шлюз, OAuth-сервіс) і використовується системою оркестрації для контролю працездатності застосунку. Структуроване логування реалізоване на основі бібліотеки `winston`: усі HTTP-запити та WebSocket-події записуються у форматі JSON з обов'язковими полями `timestamp`, `request_id` (для наскрізного трасування запиту крізь усі шари), типом події та статус-кодом.

Фінансові операції додатково фіксуються в окремому незмінному журналі (audit log) з мінімальним терміном зберігання 3 роки. Особисті дані (паролі, токени,

платіжні реквізити) виключаються з логів на рівні форматувальника. Метрики системи (кількість активних з'єднань, час відповіді, частота помилок) збираються в реальному часі, а при перевищенні порогових значень формуються автоматичні алерти, що дозволяє оперативно реагувати на деградацію сервісу.

ВИСНОВКИ

У ході виконання кваліфікаційної роботи спроектовано та реалізовано програмну систему для безконтактного замовлення їжі в ресторані з використанням QR-технологій. Створена система забезпечує повний цикл обслуговування гостя від моменту сканування QR-коду на столику до завершення оплати та залишення відгуку, замінюючи традиційну модель «гість — офіціант — кухня — офіціант — гість».

У межах роботи вирішено такі завдання:

- Виконано детальний аналіз предметної галузі та порівняльний аналіз чотирьох існуючих програмних рішень (Poster POS, OddMenu, Choice QR, Kavapp QR). За результатами аналізу виявлено ринкову прогалину: існуючі рішення або є фінансово недоступними для закладів малого формату через надлишковий функціонал, або обмежуються виключно функцією відображення меню без можливості повноцінного замовлення гостем.

- Сформовано повний комплект функціональних та нефункціональних вимог до системи. Сформульовано понад 80 функціональних вимог з унікальними ідентифікаторами, розподілених на 15 модулів, та понад 70 критеріїв приймання у форматі Given/When/Then. Результати оформлено у вигляді документа Software Requirements Specification (SRS).

- Спроектовано трирівневу клієнт-серверну архітектуру системи з використанням обґрунтовано підібраного технологічного стеку (ReactJS, Node.js, MongoDB). Розроблено схему бази даних із 23 колекціями та стратегією індексування для оптимізації типових запитів.

- Реалізовано клієнтську частину у вигляді односторінкового застосунку з підтримкою Progressive Web App, що забезпечує часткову роботу в офлайн-режимі та зберігає кошик гостя при втраті з'єднання. Інтерфейс адаптований під три типи

користувачів — гостя, персонал та адміністратора — з умовним рендерингом залежно від ролі.

– Реалізовано серверну частину з шаровою архітектурою, що включає понад 60 REST API-ендпоінтів та повноцінну систему передачі подій у реальному часі через WebSocket з моделлю кімнат та механізмом event replay. Розроблено систему автентифікації з підтримкою email+пароль та Google OAuth 2.0 для всіх типів користувачів.

– Реалізовано розширені модулі системи: аналітичний дашборд, генератор PDF-меню, систему відгуків, інтеграцію з платіжним шлюзом LiqPay для онлайн-оплати та повноцінну систему виклику офіціанта з підтримкою двох типів викликів.

– Реалізовано модель монетизації Freemium із технічним розподілом функціоналу на рівні бази даних та middleware: безкоштовний тариф включає базовий функціонал прийому та обслуговування замовлень з обмеженнями на обсяг меню та кількість акаунтів персоналу; розширений тариф знімає обмеження та активує модулі онлайн-оплати, аналітики, генератора PDF, системи відгуків та автоматизованого перекладу через Google Translate.

Усі поставлені у вступі завдання виконано в повному обсязі. Створений програмний продукт є придатним для реального впровадження в закладах громадського харчування малого та середнього формату. Модульна архітектура та чітко документована специфікація вимог забезпечують можливість подальшого розширення функціоналу без перепроєктування системи.

Перспективи подальшого розвитку. Архітектура системи спроектована з урахуванням можливості подальшого розширення функціональності. Перспективними напрямками розвитку є розширення мовної підтримки (завдяки використанню i18next додавання нової мови потребує лише перекладу ресурсних файлів без модифікації коду), розширення тем інтерфейсу через систему CSS-змінних, а також реалізація функції розділення рахунку (Split Bill), інтеграція системи бронювання столиків та програми лояльності.

Окремими перспективними напрямками є поглиблення аналітичного блоку (погодинна аналітика навантаження, прогнозування попиту, ABC-аналіз страв за прибутковістю), інтеграція зі штучним інтелектом (автоматична генерація описів страв, персоналізовані рекомендації, аналіз тональності відгуків), двостороння інтеграція з популярними POS-системами (Poster, Syrve, Servio) та розробка нативних мобільних застосунків для персоналу з push-сповіщеннями. Реалізація запропонованих напрямів забезпечить трансформацію системи з вузькоспеціалізованого SaaS-рішення на повноцінну цифрову платформу для управління закладами громадського харчування.

ПЕРЕЛІК ДЖЕРЕЛ ПОСИЛАНЬ

1. Звягінцева О. Проблеми ресторанного бізнесу в Україні у 2025 році та рішення для них [Електронний ресурс]. — URL: <https://hub.kyivstar.ua/articles/problemi-restorannogo-biznesu-v-ukrayini-u-2025-roczhi-ta-rishennya-dlya-nih> (дата звернення: 16.04.2026).
2. Островська Г., Шерстюк Р., Летун О. Аналіз ринкових тенденцій інтелектуальної автоматизації ресторанного бізнесу [Електронний ресурс]. — URL: [https://doi.org/10.32515/2663-1636.2023.10\(43\).143-155](https://doi.org/10.32515/2663-1636.2023.10(43).143-155) (дата звернення: 16.04.2026).
3. Силівейстр В. Топ 13 трендів у ресторанному бізнесі у 2026 році [Електронний ресурс]. — URL: <https://joinposter.com/ua/post/restoranni-trendy> (дата звернення: 17.04.2026).
4. Турчиняк М., Примаєв А. Вплив пандемії covid-19 на готельно-ресторанну індустрію України [Електронний ресурс]. — URL: <https://doi.org/10.32782/2524-0072/2023-47-29> (дата звернення: 17.04.2026).
5. Poster POS: офіційний сайт системи автоматизації [Електронний ресурс]. — URL: <https://joinposter.com> (дата звернення: 18.04.2026).
6. OddMenu: цифрове QR-меню для ресторанів [Електронний ресурс]. — URL: <https://oddmenu.com> (дата звернення: 18.04.2026).
7. Choice QR: система безконтактного замовлення та оплати [Електронний ресурс]. — URL: <https://choiceqr.com> (дата звернення: 18.04.2026).
8. Kavapp: система управління кав'ярнями та ресторанами [Електронний ресурс]. — URL: <https://kavapp.com> (дата звернення: 18.04.2026).
9. Бэнкс А., Порселло Е. React и Redux: функциональная веб-разработка. — Санкт-Петербург : Питер, 2019. — 336 с.
10. Бэнкер К., Бакелов П., Сэверенс Т. MongoDB в действии. — Москва : ДМК Пресс, 2020. — 432 с.

